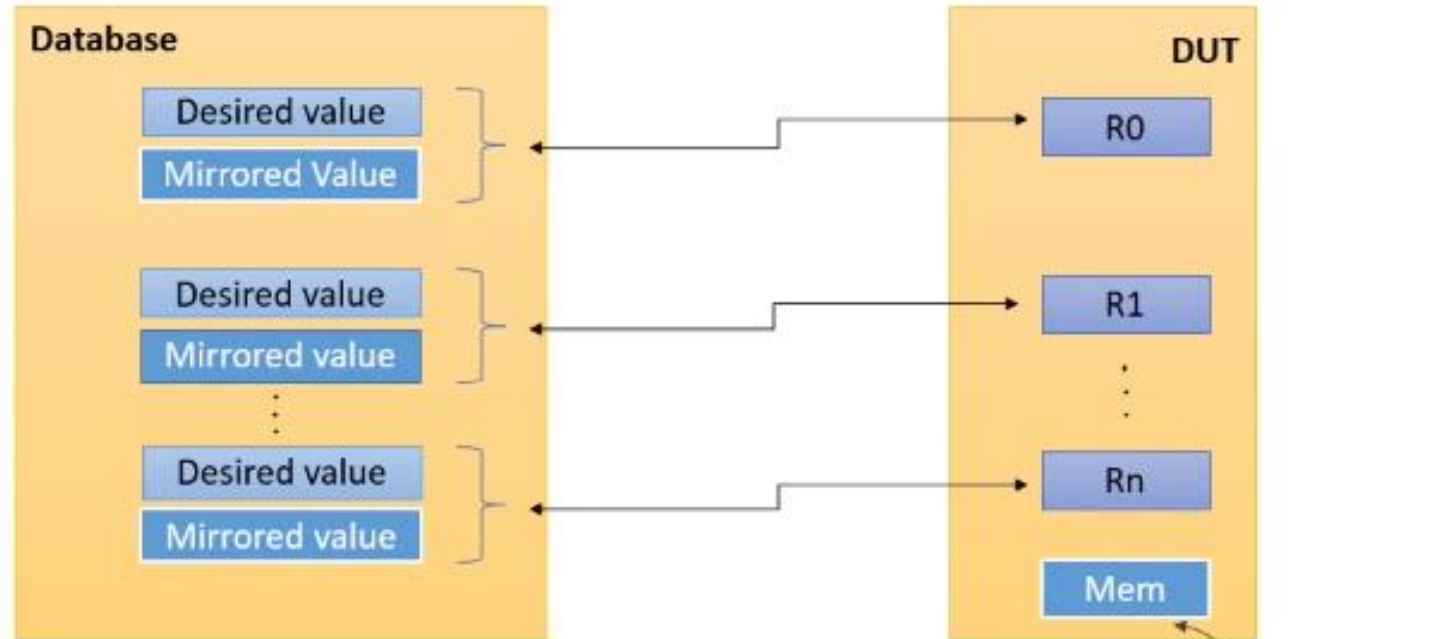


Covered Topics

□ Introduction to RAL

- ✓ What are Frontdoor and backdoor
- ✓ Types of Prediction
- ✓ Register and memory classes in RAL

Database



Desired value: - What we want in the hardware register NEXT
Mirror value: - What we think is in the hardware register NOW

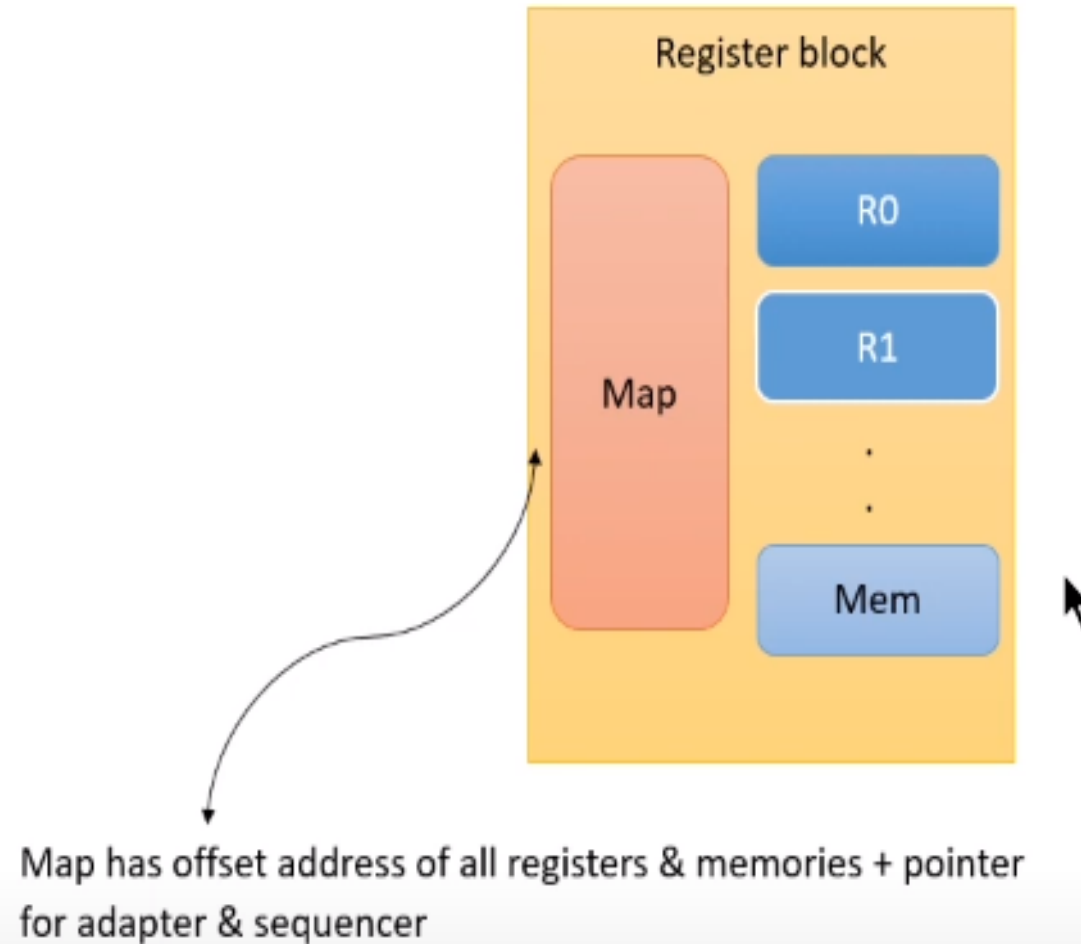
No any Desired & Mirror value for MEM

Updated by Predictor at the end of front door write & read access

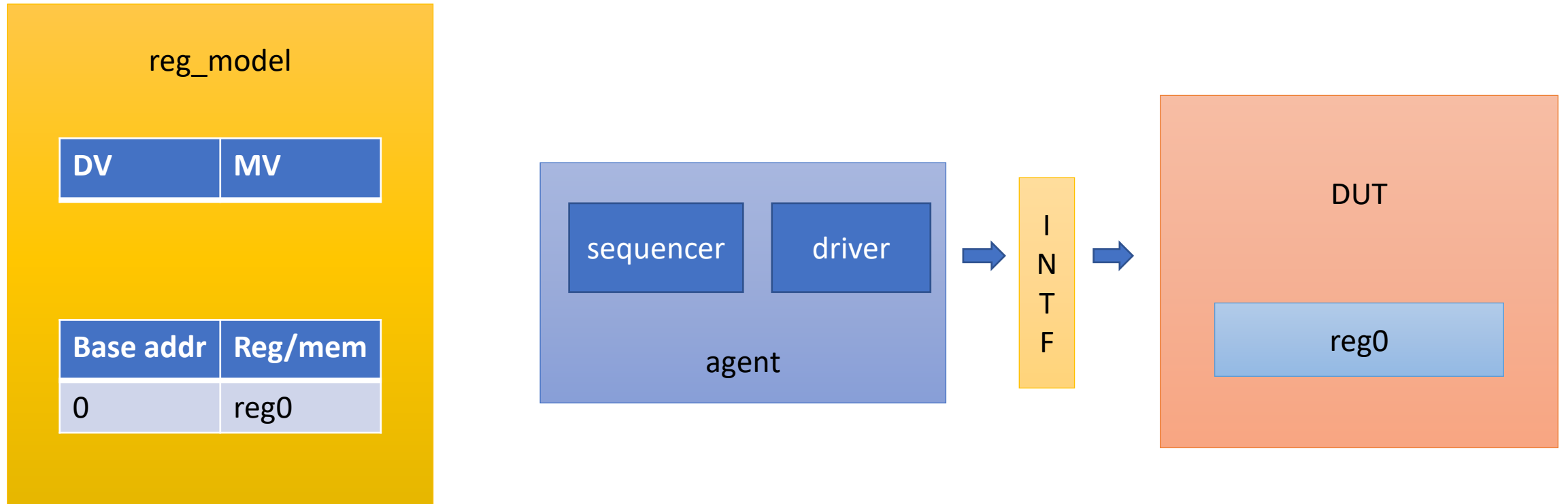
Register Model

Has 7 building block classes

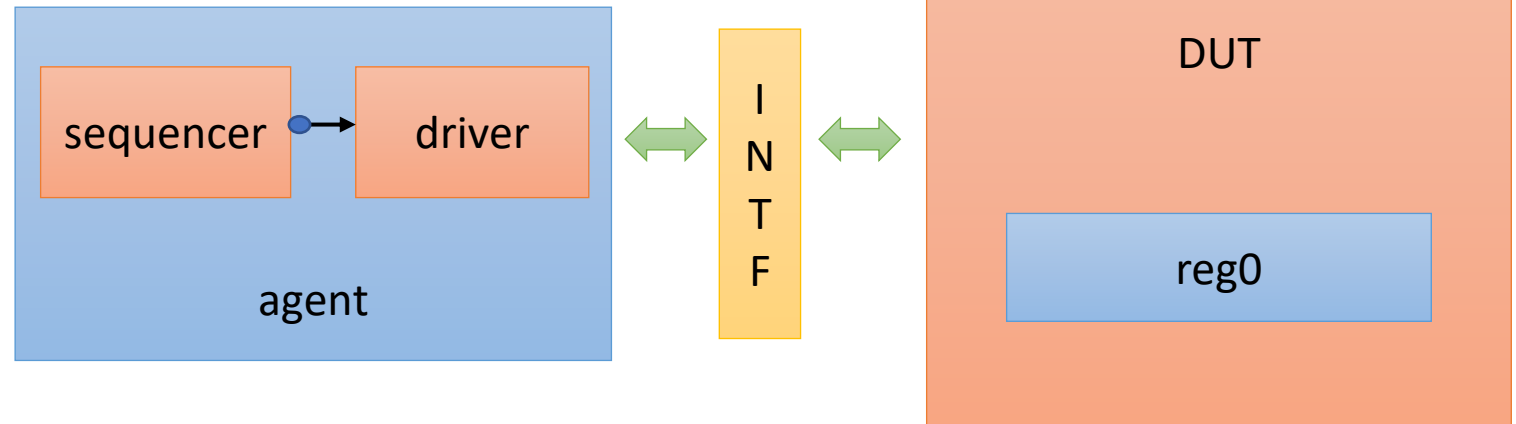
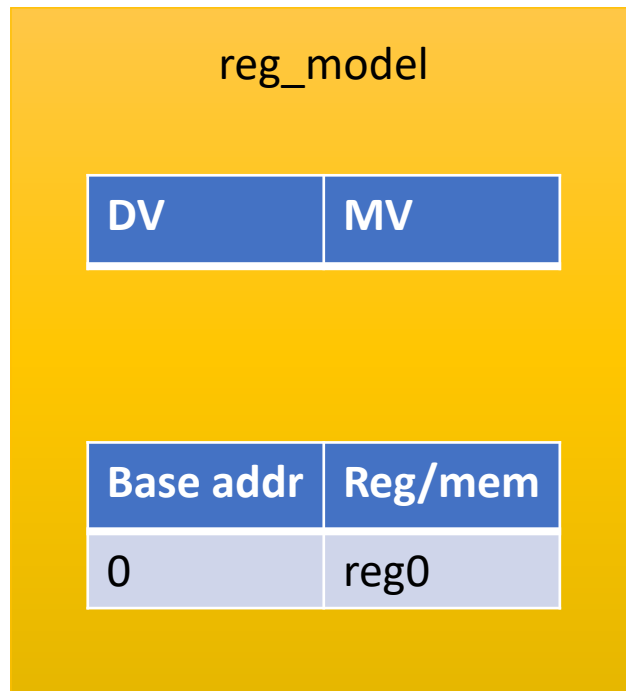
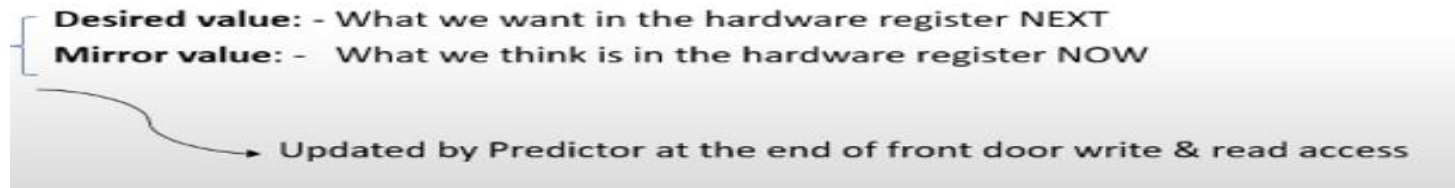
Name	Base class
Register	uvm_reg
Register field	uvm_reg_field
Register file	uvm_reg_file // optional one
Memory	uvm_mem
Register map	uvm_map
Register block	uvm_reg_block



Testbench with RAL

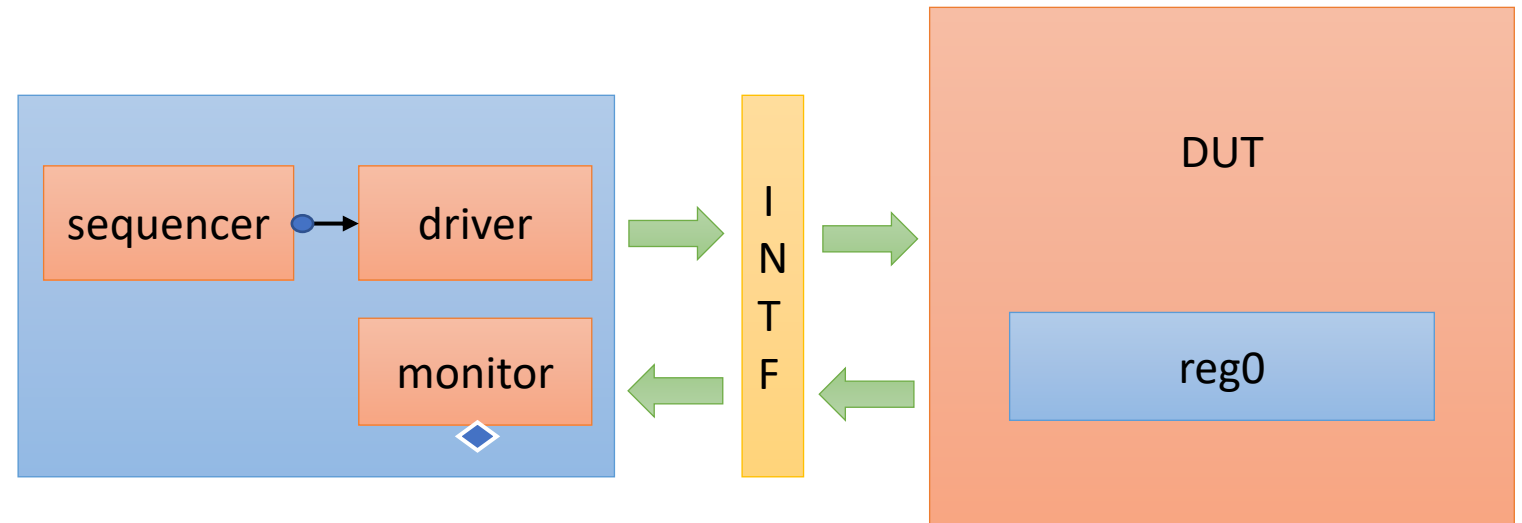
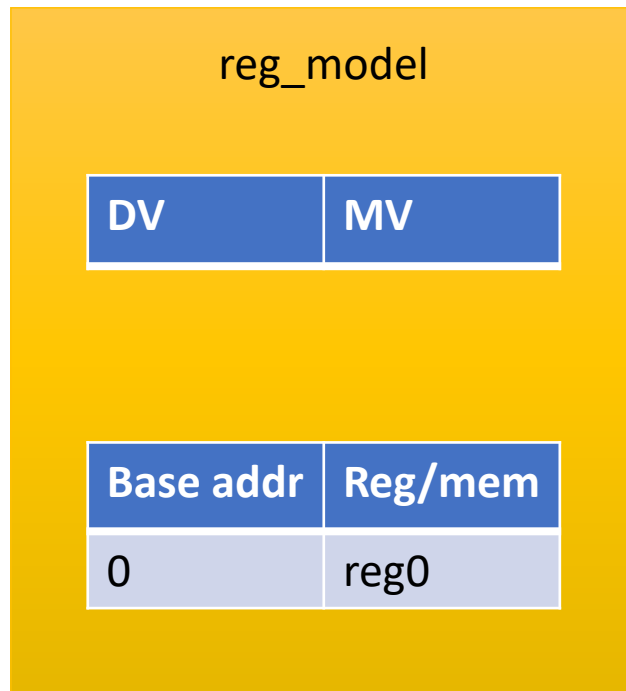
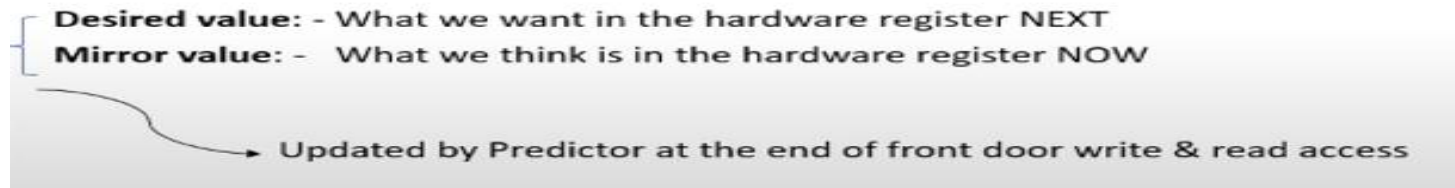


Register Abstraction Layer (RAL) w.r.p.t SV-UVM – Part 4



Testbench with RAL

Register Abstraction Layer (RAL) w.r.p.t SV-UVM – Part 4



Testbench with RAL

Adapter

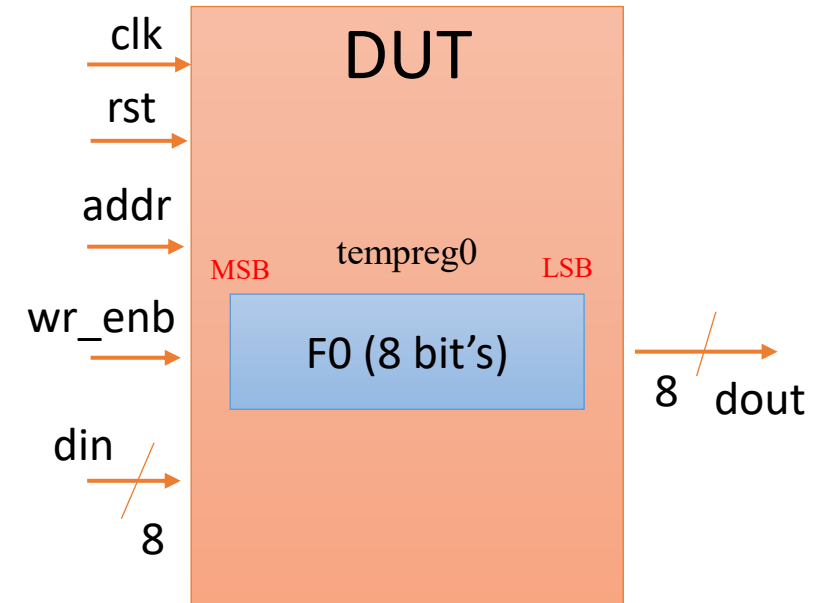
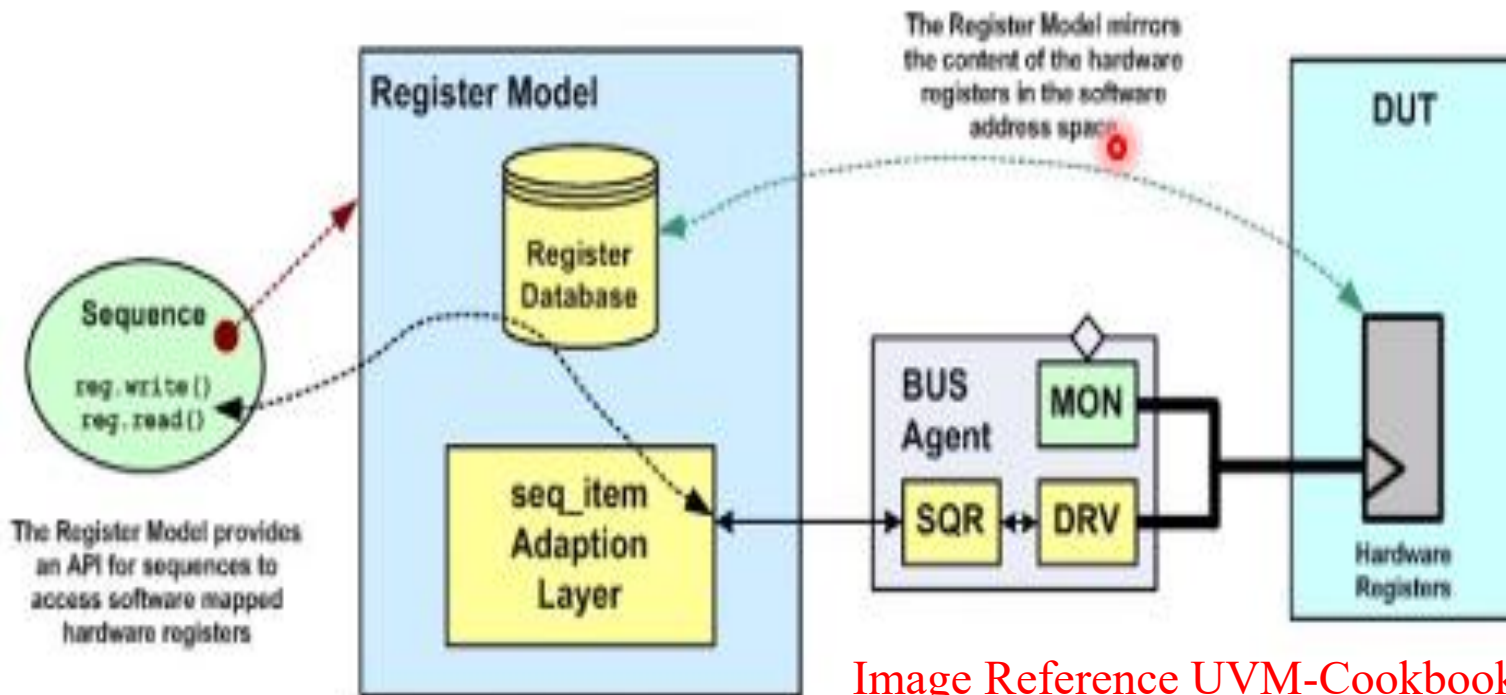
- ✓ kind
- ✓ addr
- ✓ data
- ✓ status

Reg txn

Adapter

Bus txn

- ✓ wr_enb
- ✓ addr
- ✓ din
- ✓ dout



Part 05

Munsif M. Ahmad

The standard structure of generic register txn type

uvm_reg_bus_op

Sr. No	Type	Property/Field	Description
1	uvm_reg_addr_t	addr	Address field, default to 64 bit's
2	uvm_reg_data_t	data	Write or Read data, default to 64 bit's
3	uvm_status_e	status	UVM_IS_OK, UVM_IS_NO_OK, UVM_IS_X
4	uvm_access_e	kind	UVM_READ, UVM_WRITE, UVM_BRUST_WRITE, UVM_BRUST_READ
5	uvm_reg_byte_en_t	byte_en	Byte enable
6	unsigned_int	n_bits	The number of bit's being transfer


```
// ----- adapter class -----
```

```
class adapter extends uvm_reg_adapter;
```

```
    `uvm_object_utils(adapter)
```

```
function new(input string name = "adapter");  
    super.new(name);  
endfunction: new
```

```
// reg2bus method:- convert reg txn to bus txn  
// return uvm_sequence_item
```

```
virtual function uvm_sequence_item reg2bus(const ref uvm_reg_bus_op rw); // 1 argument
```

```
    transaction itm = transaction::type_id::create("itm"); // instance of txn class and object creation
```

```
    itm.wr_enb = (rw.kind == UVM_WRITE) ? '1 : '0; // updating the bus txn with reg txn  
    itm.addr = rw.addr;
```

```
    if(itm.wr_enb) itm.din = rw.data;
```

```
    return itm;
```

```
endfunction: reg2bus
```

```
// bus2reg:- convert bus txn to reg txn
```

```
virtual function void bus2reg(uvm_sequence_item bus_item, ref uvm_reg_bus_op rw); // 2 arguments
```

```
    transaction itm; // instance of xtn class
```

```
    assert($cast(itm, bus_item)); // to get correct child class
```

```
    rw.kind = (itm.wr_enb) ? UVM_WRITE : UVM_READ; // updating the reg txn with bus txn  
    rw.data = itm.dout;  
    rw.addr = itm.addr;  
    rw.status = UVM_IS_OK;
```

```
endfunction: bus2reg
```

```
endclass: adapter
```

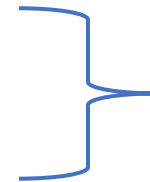
RAL Methods

- Methods before the transaction

- ✓ 1) Desired value

- ✓ a) set:- set the value for the desired variable

- ✓ b) get:- get the value of the desired variable



For register model

- ✓ 2) Mirrored value

- ✓ a) get_mirrored_value

RAL Methods

- Methods after the transaction

- ✓ 1) Frontdoor access

- ✓ a) write c) update e) predict

- ✓ b) read d) mirror f) randomize

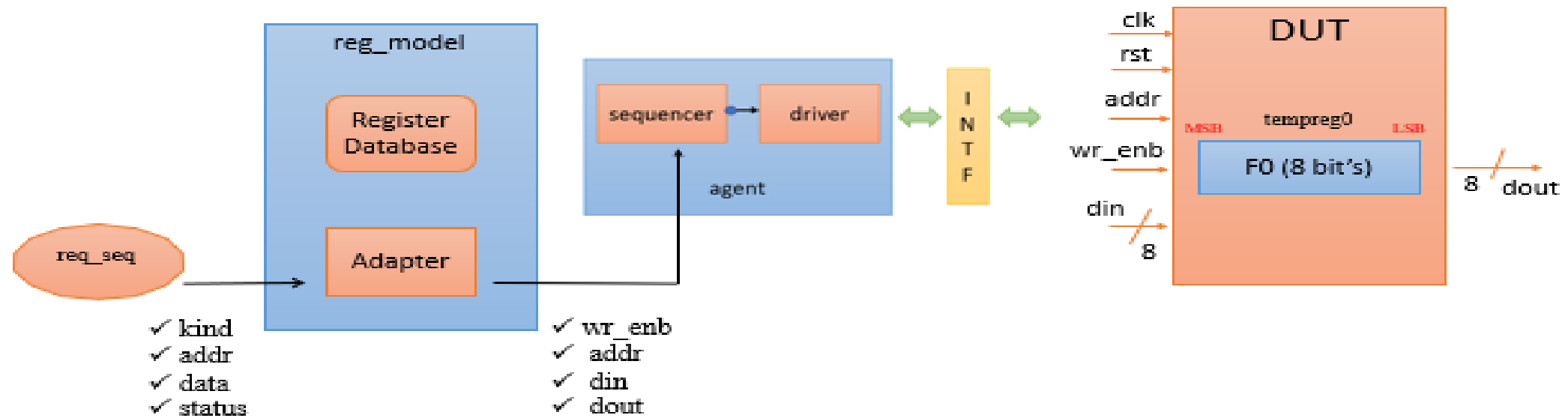
- ✓ 2) Backdoor access

- ✓ a) poke

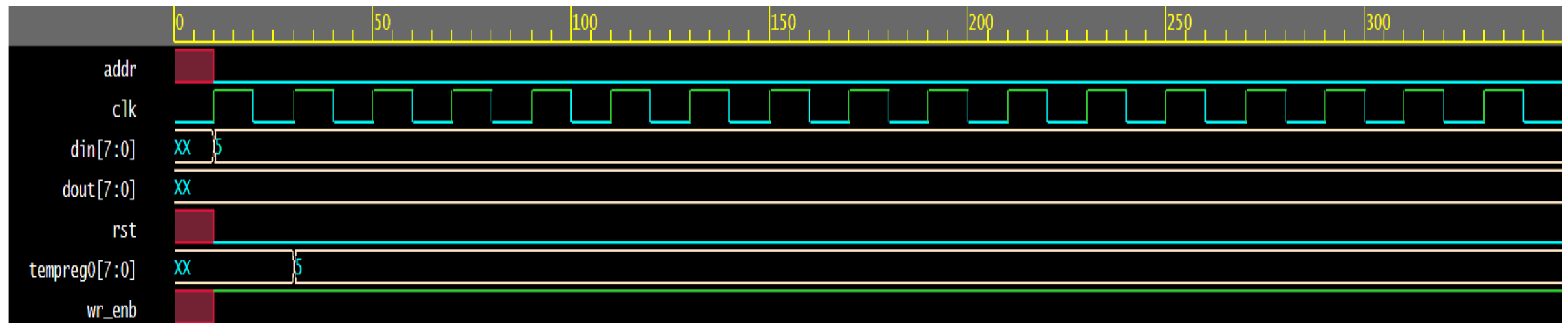
- ✓ b) peek

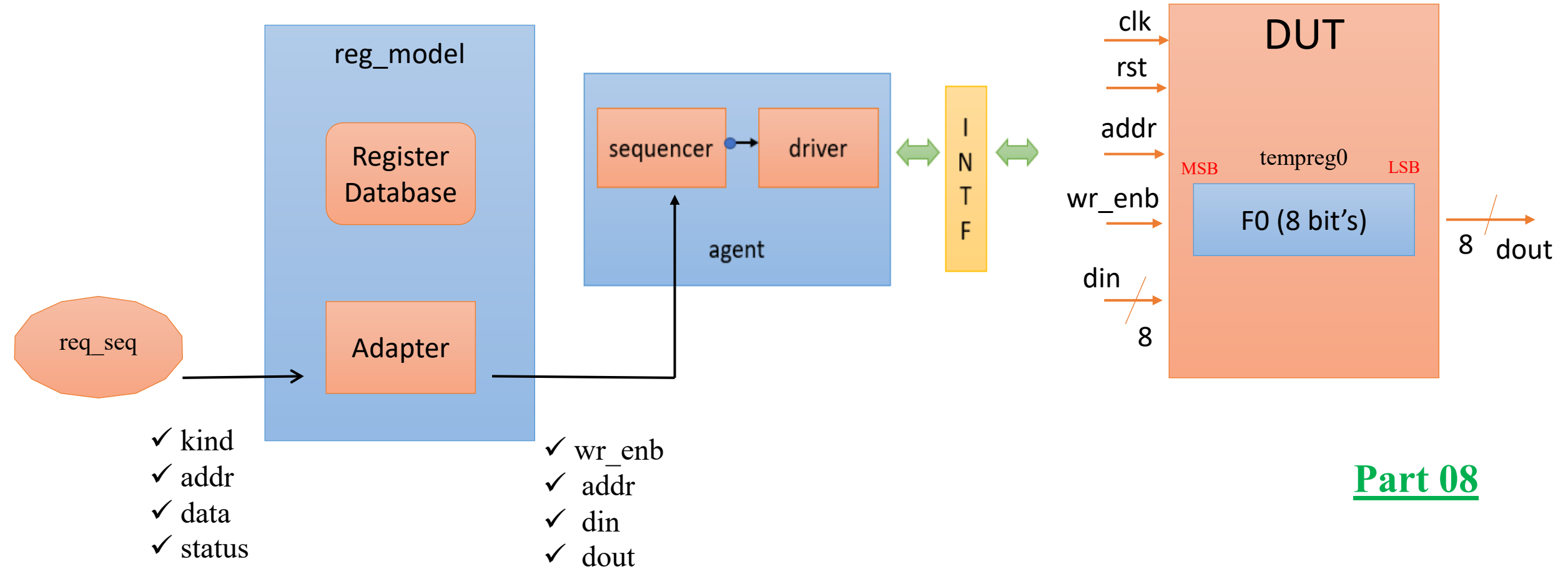
Simulation Result

```
: UVM_INFO /home/runner/testbench.sv(338) @ 0: uvm_test_top.env.agt_h.seqr@@reg_seq [register_sequence] initial Desired value is 0, & Mirrored value is 0
: UVM_INFO /home/runner/testbench.sv(355) @ 0: uvm_test_top.env.agt_h.seqr@@reg_seq [register_sequence] After set method Desired value is 10, & Mirrored value is 0
: UVM_INFO /home/runner/testbench.sv(125) @ 50: uvm_test_top.env.agt_h.drv [driver] DATA WRITE wr_data 5
: UVM_INFO /home/runner/testbench.sv(365) @ 50: uvm_test_top.env.agt_h.seqr@@reg_seq [register_sequence] After write method Desired value is 5, & Mirrored value is 5
: UVM_INFO /home/build/vlib1/vlib/uvm-1.2/src/base/uvm_objection.svh(1271) @ 350: reporter [TEST_DONE] 'run' phase is ready to proceed to the 'extract' phase
: UVM_INFO /home/runner/testbench.sv(556) @ 350: uvm_test_top [test] -----
: UVM_INFO /home/runner/testbench.sv(557) @ 350: uvm_test_top [test] ----- TEST PASS -----
: UVM_INFO /home/runner/testbench.sv(558) @ 350: uvm_test_top [test] -----
: UVM_INFO /home/build/vlib1/vlib/uvm-1.2/src/base/uvm_report_server.svh(869) @ 350: reporter [UVM/REPORT/SERVER]
```

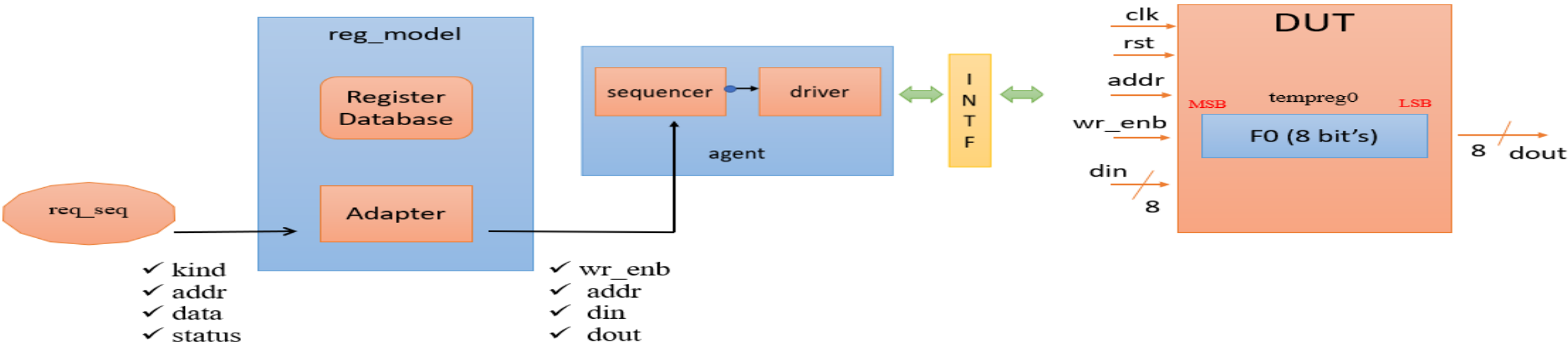


Part 07

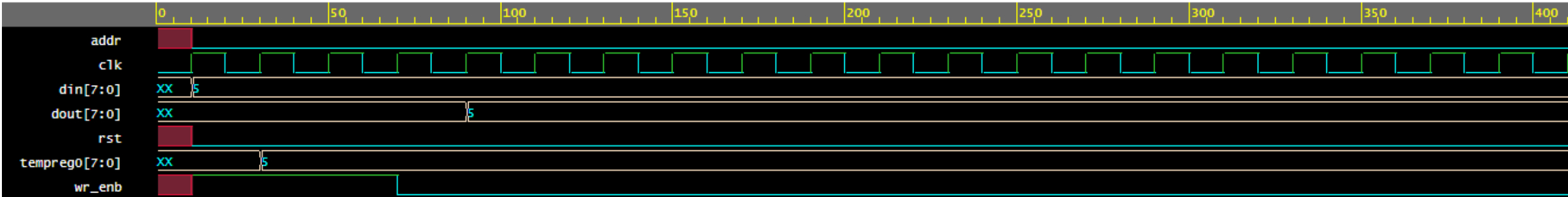




Part 08

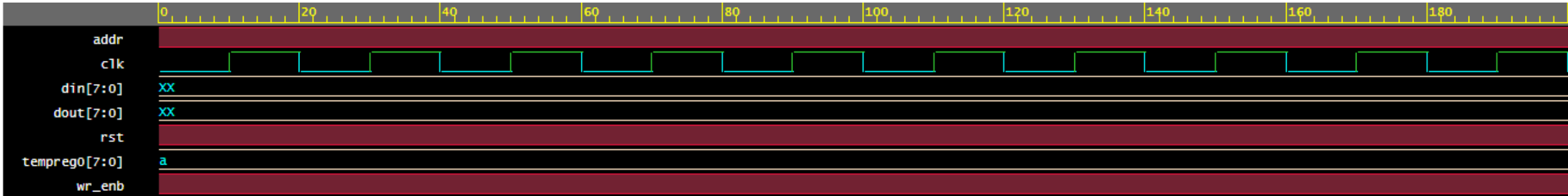


Part 08



W/F for frontdoor write and read methods

W/F for backdoor poke and peek methods



RAL Methods

- Methods after the transaction

- ✓ 1) Frontdoor access

- ✓ a) write c) update e) predict

- ✓ b) read d) mirror f) randomize

For write and read methods (Note:- comment the clk generation and observe)

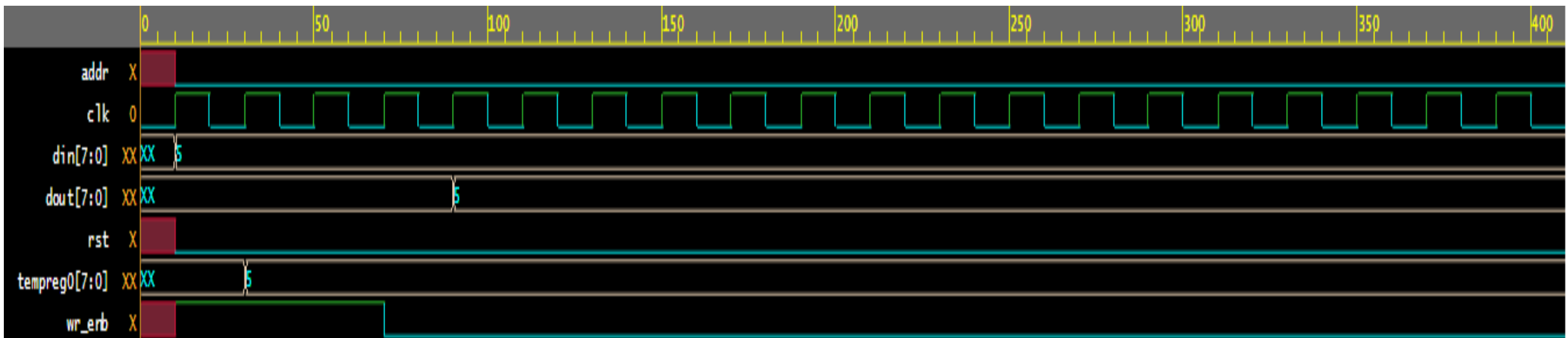
<https://edaplayground.com/x/EGvc>

Frontdoor write and read methods

```

JVM_INFO /home/runner/testbench.sv(127) @ 50: uvm_test_top.env.agt_h.drv [driver] DATA WRITE wr_data 5
JVM_INFO /home/runner/testbench.sv(343) @ 50: uvm_test_top.env.agt_h.seqr@@reg_seq [register_sequence] After write method Desired value is 5, & Mirrored value is 5
JVM_INFO /home/runner/testbench.sv(135) @ 110: uvm_test_top.env.agt_h.drv [driver] DATA READ rd_data 5
JVM_INFO /home/runner/testbench.sv(356) @ 110: uvm_test_top.env.agt_h.seqr@@reg_seq [register_sequence] After read method Desired value is 5, & Mirrored value is 5, read_out is 5
JVM_INFO /home/build/vlib1/vlib/uvm-1.2/src/base/uvm_objection.svh(1271) @ 410: reporter [TEST_DONE] 'run' phase is ready to proceed to the 'extract' phase
JVM_INFO /home/runner/testbench.sv(549) @ 410: uvm_test_top [test] -----
JVM_INFO /home/runner/testbench.sv(550) @ 410: uvm_test_top [test] ----- TEST PASS -----
JVM_INFO /home/runner/testbench.sv(551) @ 410: uvm_test_top [test] -----
JVM_INFO /home/build/vlib1/vlib/uvm-1.2/src/base/uvm_report_server.svh(869) @ 410: reporter [UVM/REPORT/SERVER]
--- UVM Report Summary ---

```



RAL Methods

- Methods after the transaction

- ✓ 2) Backdoor access

- ✓ a) poke

- ✓ b) peek

<https://edaplayground.com/x/PLKh>

Backdoor poke and peek methods

```
***** backdoor write i.e poke *****
```

```
INFO /home/runner/testbench.sv(390) @ 0: uvm_test_top.env.agt_h.seqr@@reg_seq [reg_sequence] After backdoor poke(poke - backdoor write) method, Desired value is 10, Mirrored value is 10
```

```
***** backdoor read i.e peek *****
```

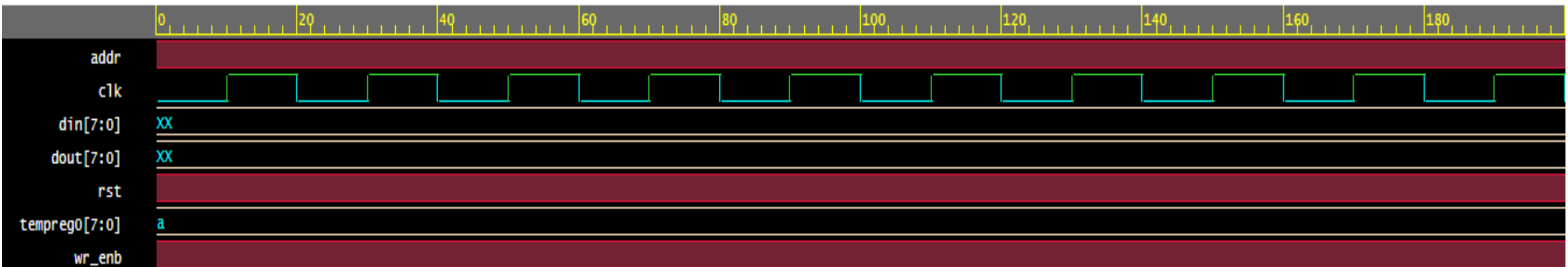
```
INFO /home/runner/testbench.sv(407) @ 0: uvm_test_top.env.agt_h.seqr@@reg_seq [reg_sequence] After backdoor peek(peek - backdoor read) method, Desired value is 10, Mirrored value is 10, read_out is 10
```

```
INFO /home/build/vlib1/vlib/uvm-1.2/src/base/uvm_objection.svh(1271) @ 200: reporter [TEST_DONE] 'run' phase is ready to proceed to the 'extract' phase
```

```
INFO /home/runner/testbench.sv(597) @ 200: uvm_test_top [test] -----
```

```
INFO /home/runner/testbench.sv(598) @ 200: uvm_test_top [test] ----- TEST PASS -----
```

```
INFO /home/runner/testbench.sv(599) @ 200: uvm_test_top [test] -----
```



RAL Methods

- Methods after the transaction

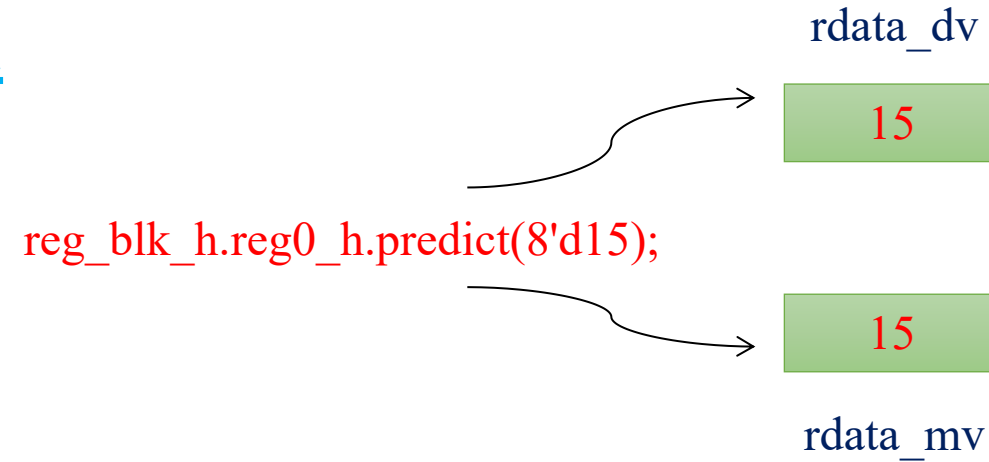
- ✓ 1) Frontdoor access

- ✓ a) write c) update e) predict

- ✓ b) read d) mirror f) randomize

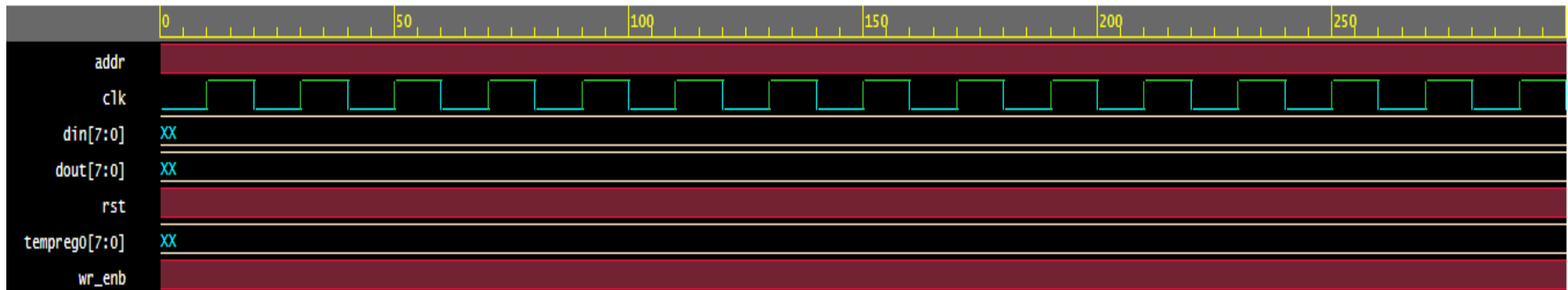
predict method

<https://edaplayground.com/x/TsLi>



Simulation Result for predict method

```
'home/runner/testbench.sv(343) @ 0: uvm_test_top.env.agt_h.seqr@@reg_seq [register_sequence] initial Desired value is 0, & Mirrored value is 0
/home/runner/testbench.sv(358) @ 0: uvm_test_top.env.agt_h.seqr@@reg_seq [register_sequence] After predict method Desired value is 15, & Mirrored value is 15
/home/build/vlib1/vlib/uvm-1.2/src/base/uvm_objection.svh(1271) @ 300: reporter [TEST_DONE] 'run' phase is ready to proceed to the 'extract' phase
/home/runner/testbench.sv(553) @ 300: uvm_test_top [test] -----
/home/runner/testbench.sv(554) @ 300: uvm_test_top [test] ----- TEST PASS -----
/home/runner/testbench.sv(555) @ 300: uvm_test_top [test] -----
/home/build/vlib1/vlib/uvm-1.2/src/base/uvm_report_server.svh(869) @ 300: reporter [UVM/REPORT/SERVER]
report Summary
```



W/F for predict method

RAL API Methods

✓ 1) Frontdoor Methods

a) write b) read c) predict d) mirror e) update

✓ 2) Backdoor Methods

a) poke b) peek

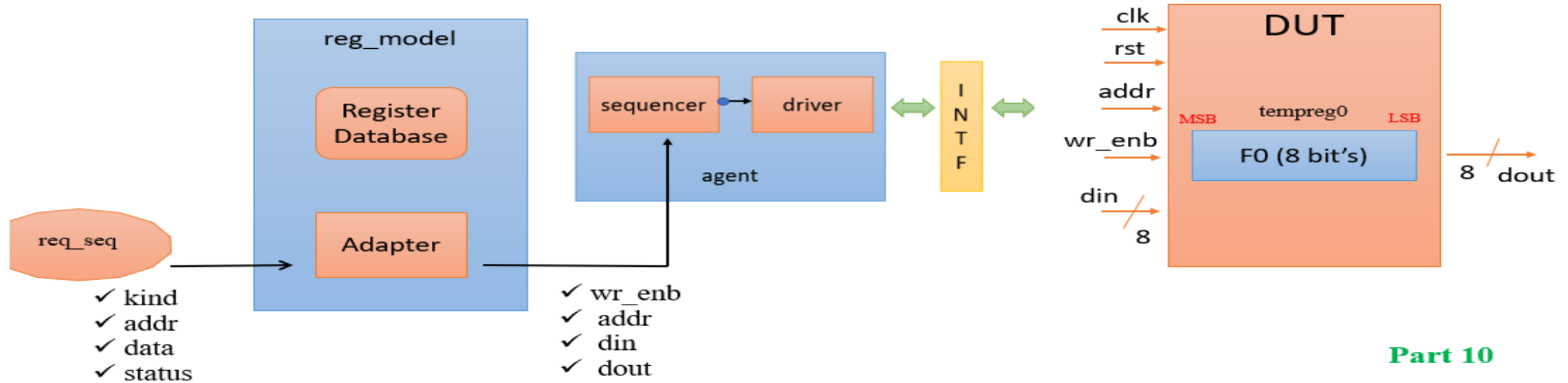
✓ 3) Other Methods

a) set b) get c) get_mirrored_value d) randomize e) reset

Simulation Result for mirror method

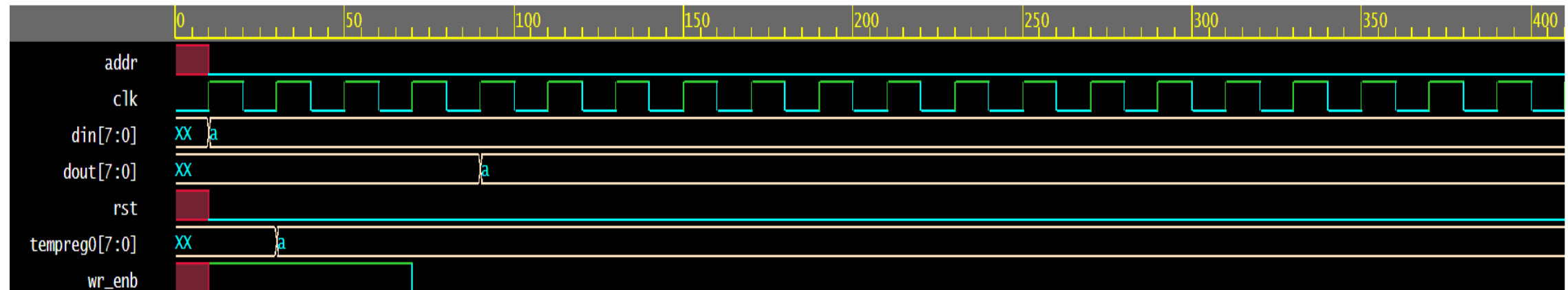
```
VEL: UVM_INFO /home/runner/testbench.sv(128) @ 50: uvm_test_top.env.agt_h.drv [driver] DATA WRITE wr_data 10
VEL: UVM_INFO /home/runner/testbench.sv(338) @ 50: uvm_test_top.env.agt_h.seqr@@reg_seq [register_sequence] After write method Desired value is 10, & Mirrored value is 10
VEL: UVM_INFO /home/runner/testbench.sv(353) @ 50: uvm_test_top.env.agt_h.seqr@@reg_seq [register_sequence] After predict method Desired value is 5, & Mirrored value is 5
VEL: UVM_INFO /home/runner/testbench.sv(136) @ 110: uvm_test_top.env.agt_h.drv [driver] DATA READ rd_data 10
VEL: UVM_ERROR /home/build/vlib1/vlib/uvm-1.2/src/reg/uvm_reg.svh(2906) @ 110: reporter [RegModel] Register "reg_blk_h.reg0_h" value read from DUT (0x000000000000000a) does not match mirrored value (0x0000000000000005)
VEL: UVM_INFO /home/build/vlib1/vlib/uvm-1.2/src/reg/uvm_reg.svh(2925) @ 110: reporter [RegModel] Field f0 (reg_blk_h.reg0_h[7:0]) mismatch read=8'h a mirrored=8'h 5
VEL: UVM_INFO /home/runner/testbench.sv(369) @ 110: uvm_test_top.env.agt_h.seqr@@reg_seq [register_sequence] After mirror method Desired value is 10, & Mirrored value is 10
VEL: UVM_INFO /home/build/vlib1/vlib/uvm-1.2/src/base/uvm_objection.svh(1271) @ 410: reporter [TEST_DONE] 'run' phase is ready to proceed to the 'extract' phase
VEL: UVM_INFO /home/runner/testbench.sv(555) @ 410: uvm_test_top [test] -----
VEL: UVM_INFO /home/runner/testbench.sv(556) @ 410: uvm_test_top [test] ----- TEST FAIL -----
VEL: UVM_INFO /home/runner/testbench.sv(557) @ 410: uvm_test_top [test] -----
```

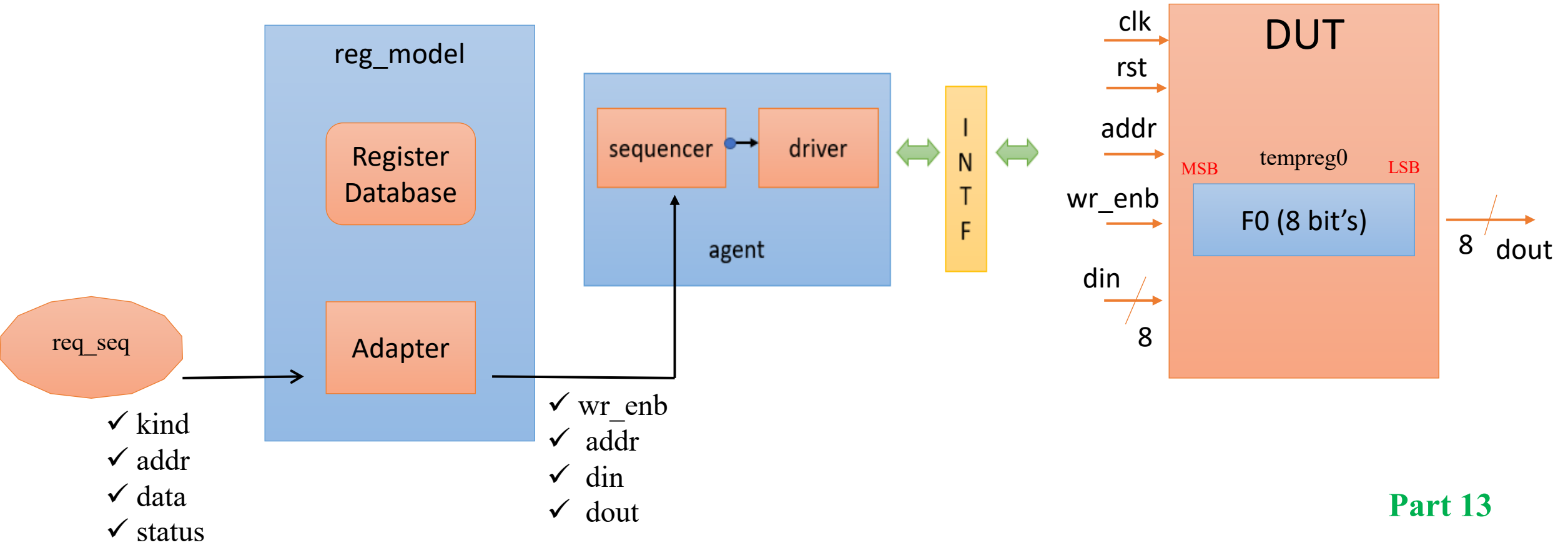
mirror method w.r.p.t SV-UVM RAL



Part 10

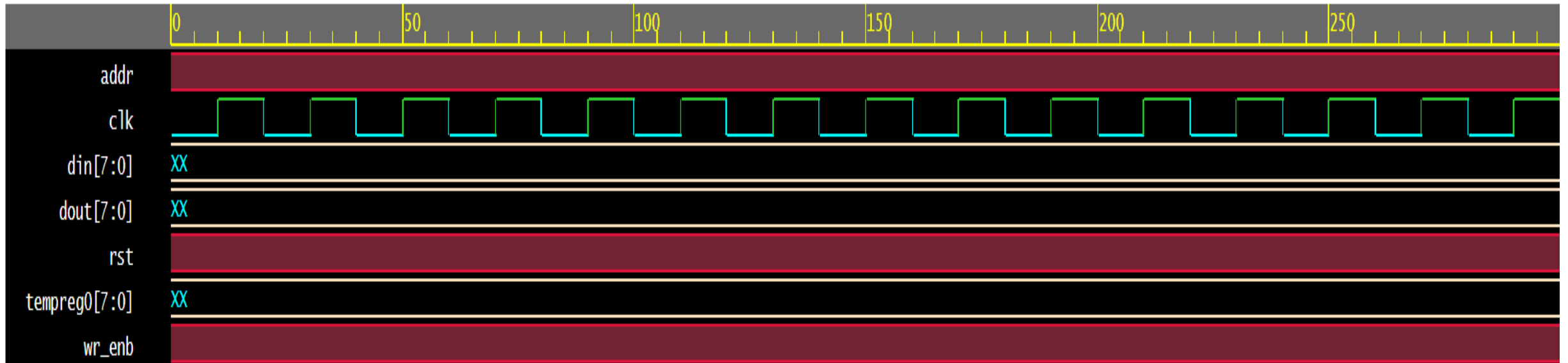
Munsif M. Ahmad



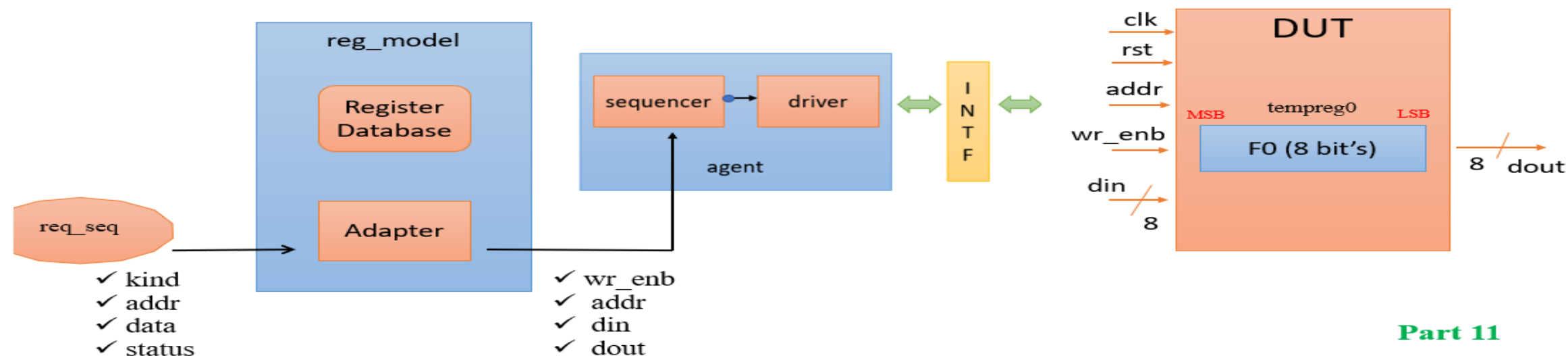


Part 13

Predict method then update method

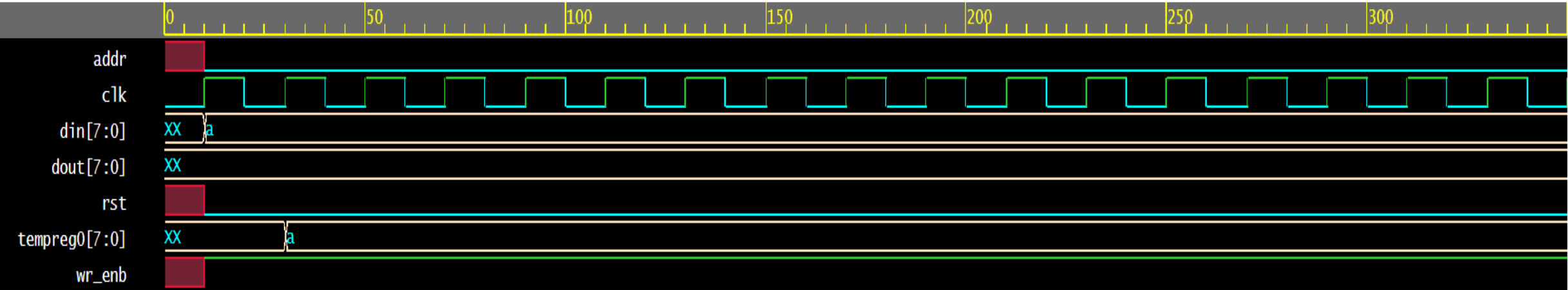


update method w.r.p.t SV-UVM RAL

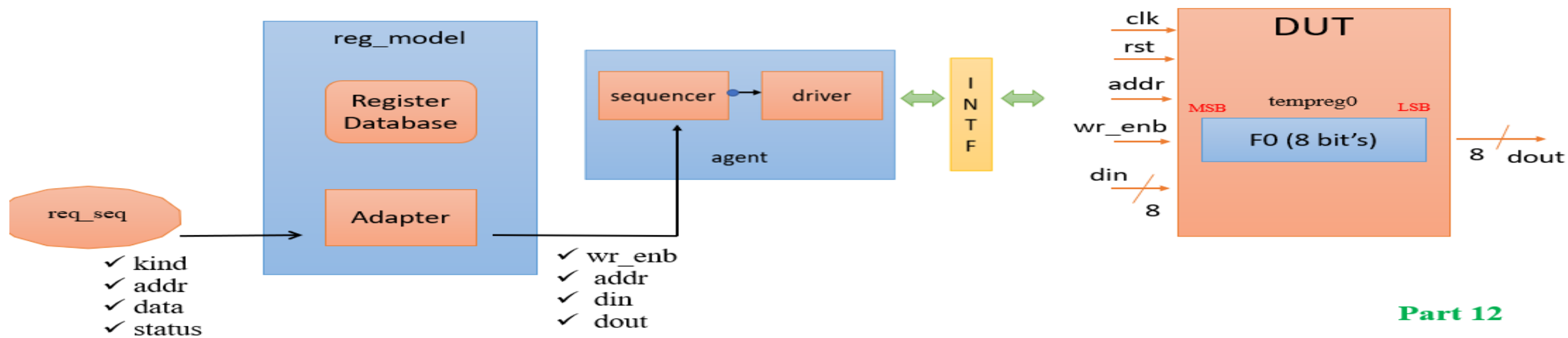


Part 11

Munsif M. Ahmad

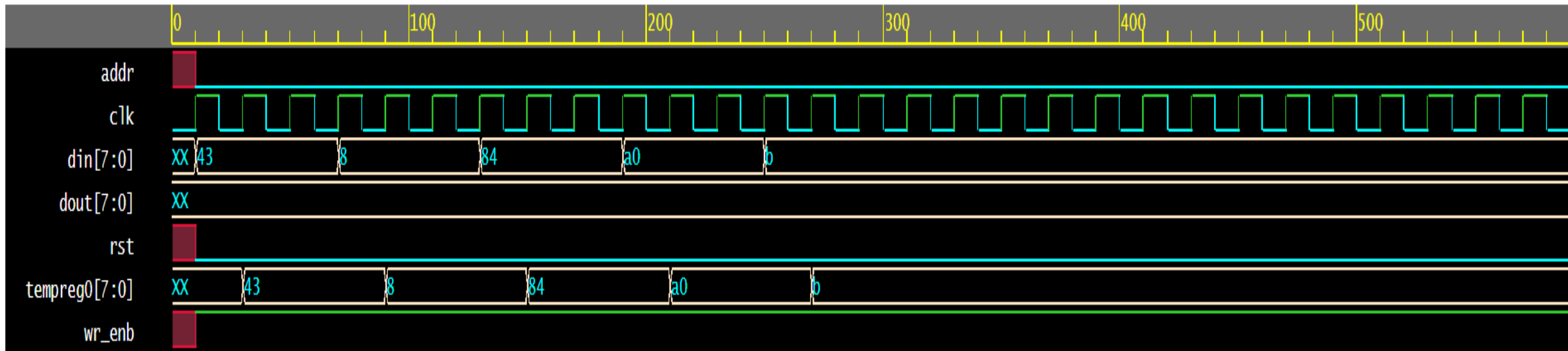


randomize method w.r.p.t SV-UVM RAL



Part 12

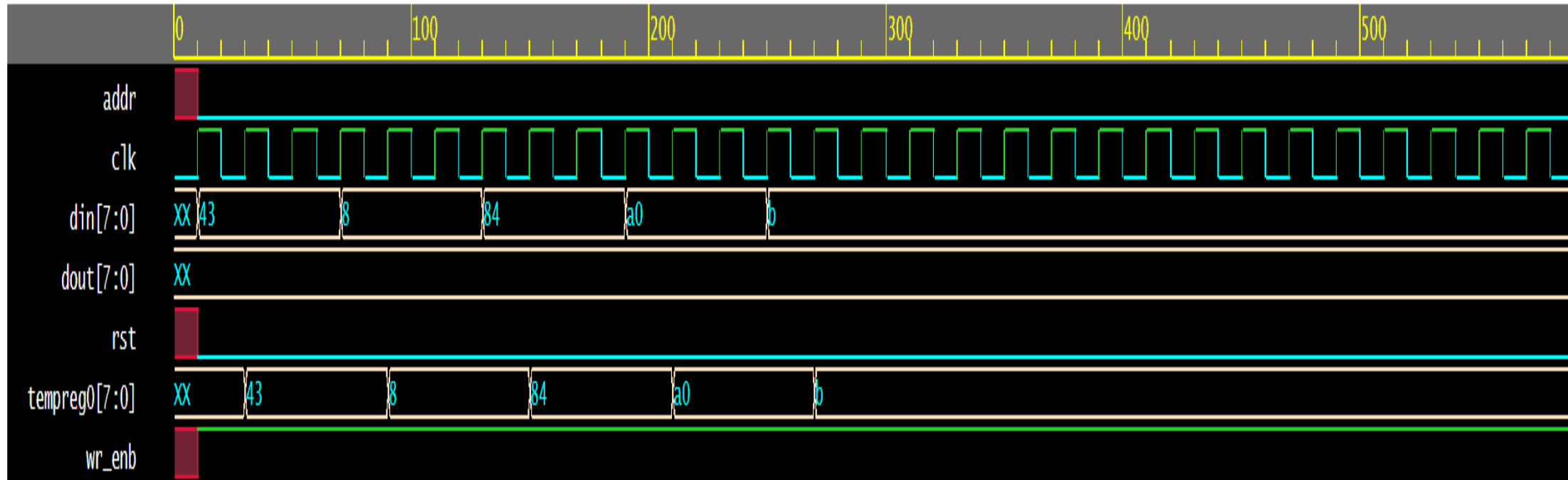
Munsif M. Ahmad



randomize method simulation result

```
# KERNEL: UVM_INFO /home/runner/testbench.sv(136) @ 50: uvm_test_top.env.agt_h.drv [driver] DATA WRITE wr_data 43
# KERNEL: UVM_INFO /home/runner/testbench.sv(351) @ 50: uvm_test_top.env.agt_h.seqr@@reg_seq [register_sequence] After randomize method random values are 0000000000000043
# KERNEL: UVM_INFO /home/runner/testbench.sv(358) @ 50: uvm_test_top.env.agt_h.seqr@@reg_seq [register_sequence] After write method Desired value is 43, & Mirrored value is 43
# KERNEL: UVM_INFO /home/runner/testbench.sv(136) @ 110: uvm_test_top.env.agt_h.drv [driver] DATA WRITE wr_data 08
# KERNEL: UVM_INFO /home/runner/testbench.sv(351) @ 110: uvm_test_top.env.agt_h.seqr@@reg_seq [register_sequence] After randomize method random values are 0000000000000008
# KERNEL: UVM_INFO /home/runner/testbench.sv(358) @ 110: uvm_test_top.env.agt_h.seqr@@reg_seq [register_sequence] After write method Desired value is 08, & Mirrored value is 08
# KERNEL: UVM_INFO /home/runner/testbench.sv(136) @ 170: uvm_test_top.env.agt_h.drv [driver] DATA WRITE wr_data 84
# KERNEL: UVM_INFO /home/runner/testbench.sv(351) @ 170: uvm_test_top.env.agt_h.seqr@@reg_seq [register_sequence] After randomize method random values are 0000000000000084
# KERNEL: UVM_INFO /home/runner/testbench.sv(358) @ 170: uvm_test_top.env.agt_h.seqr@@reg_seq [register_sequence] After write method Desired value is 84, & Mirrored value is 84
# KERNEL: UVM_INFO /home/runner/testbench.sv(136) @ 230: uvm_test_top.env.agt_h.drv [driver] DATA WRITE wr_data a0
# KERNEL: UVM_INFO /home/runner/testbench.sv(351) @ 230: uvm_test_top.env.agt_h.seqr@@reg_seq [register_sequence] After randomize method random values are 00000000000000a0
# KERNEL: UVM_INFO /home/runner/testbench.sv(358) @ 230: uvm_test_top.env.agt_h.seqr@@reg_seq [register_sequence] After write method Desired value is a0, & Mirrored value is a0
# KERNEL: UVM_INFO /home/runner/testbench.sv(136) @ 290: uvm_test_top.env.agt_h.drv [driver] DATA WRITE wr_data 0b
# KERNEL: UVM_INFO /home/runner/testbench.sv(351) @ 290: uvm_test_top.env.agt_h.seqr@@reg_seq [register_sequence] After randomize method random values are 000000000000000b
# KERNEL: UVM_INFO /home/runner/testbench.sv(358) @ 290: uvm_test_top.env.agt_h.seqr@@reg_seq [register_sequence] After write method Desired value is 0b, & Mirrored value is 0b
# KERNEL: UVM_INFO /home/build/vlib1/vlib/uvm-1.2/src/base/uvm_objection.svh(1271) @ 590: reporter [TEST_DONE] 'run' phase is ready to proceed to the 'extract' phase
# KERNEL: UVM_INFO /home/runner/testbench.sv(552) @ 590: uvm_test_top [test] -----
# KERNEL: UVM_INFO /home/runner/testbench.sv(553) @ 590: uvm_test_top [test] ----- TEST PASS -----
# KERNEL: UVM_INFO /home/runner/testbench.sv(554) @ 590: uvm_test_top [test] -----
# KERNEL: UVM_INFO /home/build/vlib1/vlib/uvm-1.2/src/base/uvm_report_server.svh(860) @ 590: reporter [UVM_REPORT_C550501]
```

randomize method simulation result



W/F for the randomize method

RAL API Methods

✓ 1) Frontdoor Methods

a) write b) read c) predict d) mirror e) update

✓ 2) Backdoor Methods

a) poke b) peek

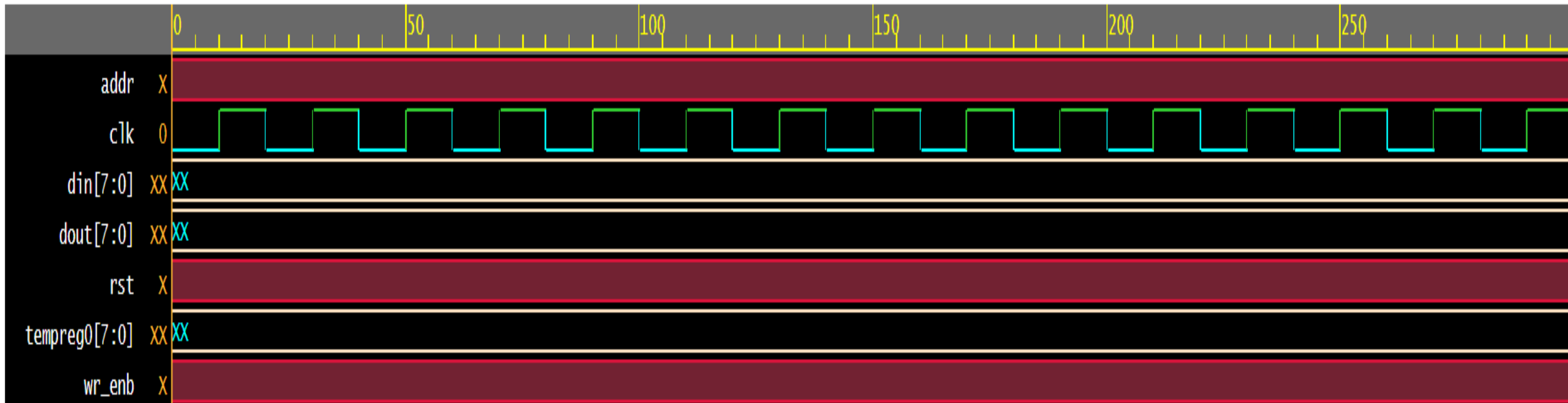
✓ 3) Other Methods

a) set b) get c) get_mirrored_value d) randomize e) reset

reset method simulation result

```
# KERNEL:
# KERNEL: UVM_INFO /home/runner/testbench.sv(367) @ 0: uvm_test_top.env.agt_h.seqr@@reg_seq [register_sequence] After has_reset method, has_rst value is 1
# KERNEL: UVM_INFO /home/runner/testbench.sv(372) @ 0: uvm_test_top.env.agt_h.seqr@@reg_seq [register_sequence] After get_reset method, get_rst value is 10
# KERNEL: ****-----**** get the dv and mv before applying the reset ****-----****
# KERNEL: UVM_INFO /home/runner/testbench.sv(390) @ 0: uvm_test_top.env.agt_h.seqr@@reg_seq [register_sequence] Before reset method Desired value is 0, & Mirrored value is 0
# KERNEL: ****-----**** get the dv and mv after applying the reset ****-----****
# KERNEL: UVM_INFO /home/runner/testbench.sv(405) @ 0: uvm_test_top.env.agt_h.seqr@@reg_seq [register_sequence] After reset method Desired value is 10, & Mirrored value is 10
# KERNEL: ****-----**** get the dv and mv after set_reset ****-----****
# KERNEL: UVM_INFO /home/runner/testbench.sv(421) @ 0: uvm_test_top.env.agt_h.seqr@@reg_seq [register_sequence] After set_reset method Desired value is 15, & Mirrored value is 15
# KERNEL: UVM_INFO /home/build/vlib1/vlib/uvm-1.2/src/base/uvm_objection.svh(1271) @ 300: reporter [TEST_DONE] 'run' phase is ready to proceed to the 'extract' phase
# KERNEL: UVM_INFO /home/runner/testbench.sv(614) @ 300: uvm_test_top [test] -----
# KERNEL: UVM_INFO /home/runner/testbench.sv(615) @ 300: uvm_test_top [test] ----- TEST PASS -----
# KERNEL: UVM_INFO /home/runner/testbench.sv(616) @ 300: uvm_test_top [test] -----
# KERNEL: UVM_INFO /home/build/vlib1/vlib/uvm-1.2/src/base/uvm_report_server.svh(869) @ 300: reporter [UVM/REPORT/SERVER]
```

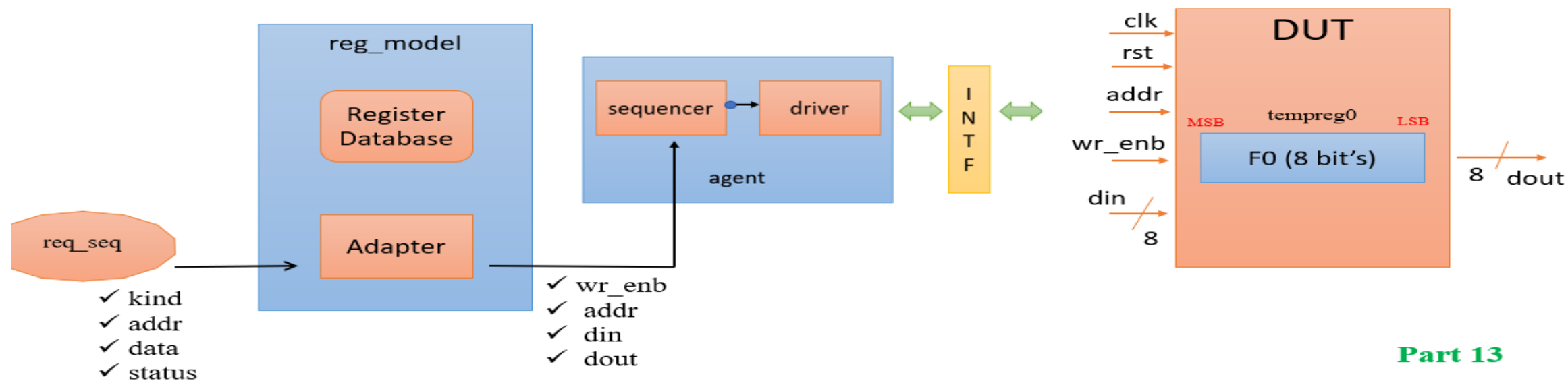

reset method simulation result



W/F for the reset method

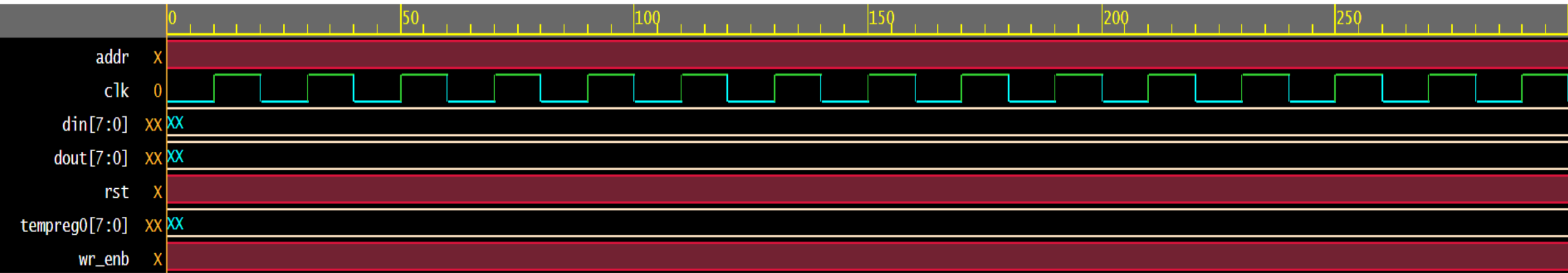
top

reset method w.r.p.t SV-UVM RAL



Part 13

Munsif M. Ahmad

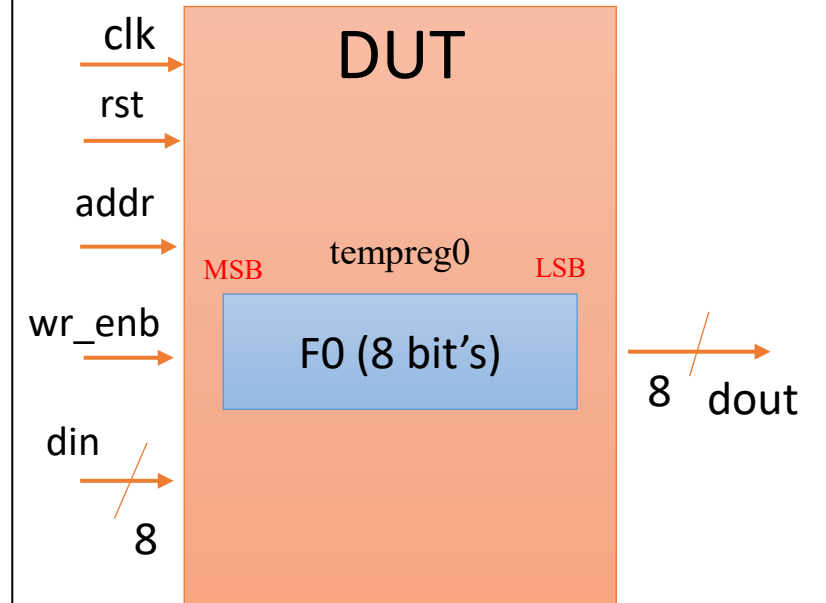
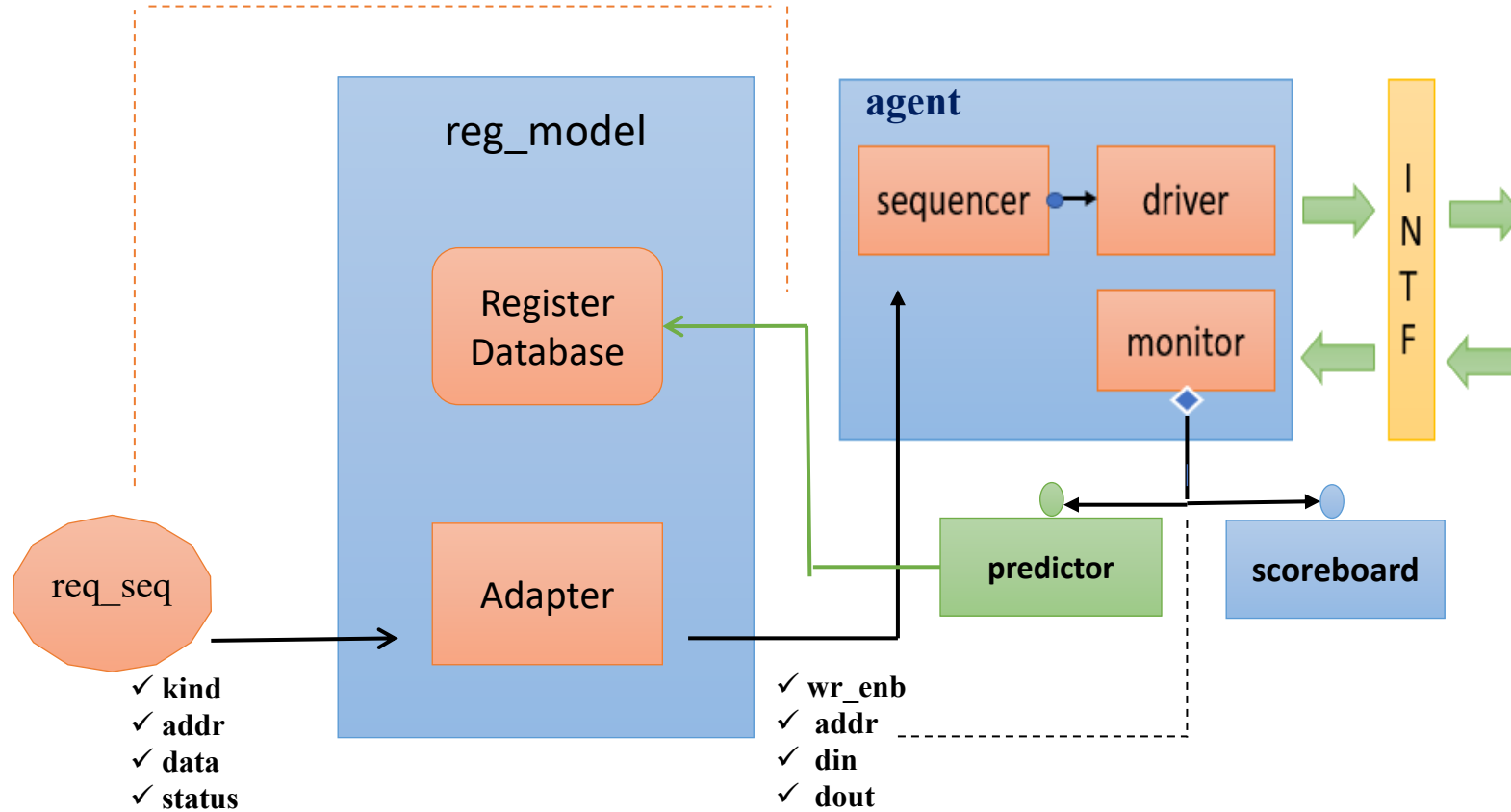


W/F for the reset method

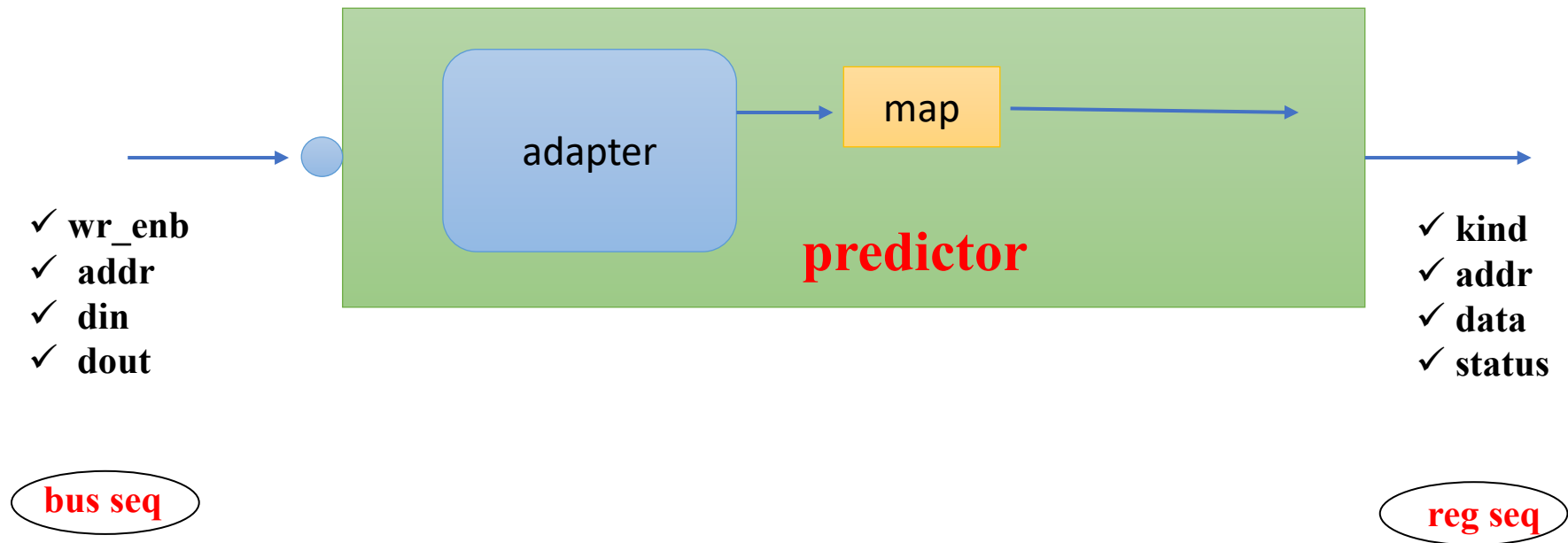
top

test

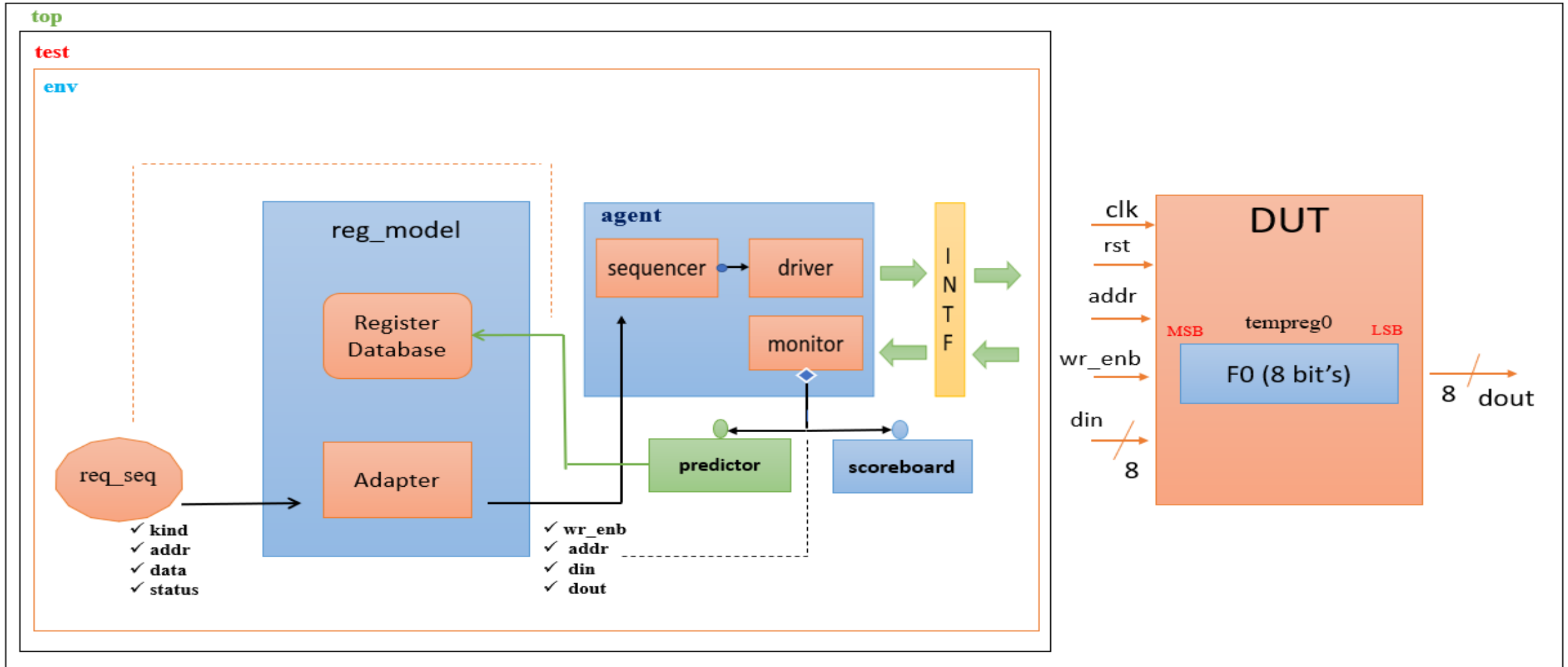
env



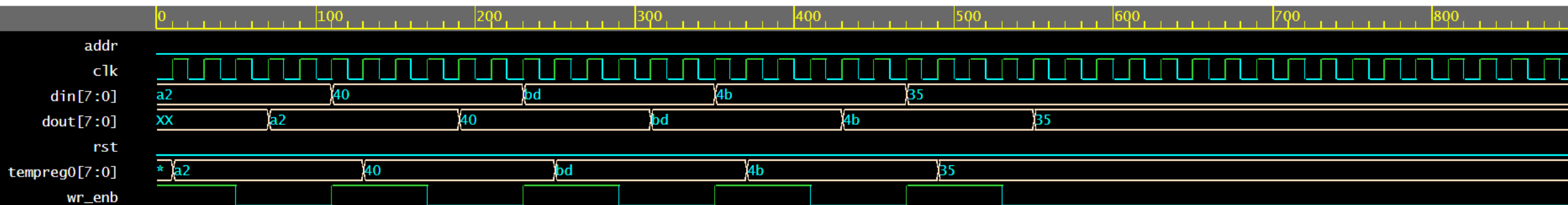
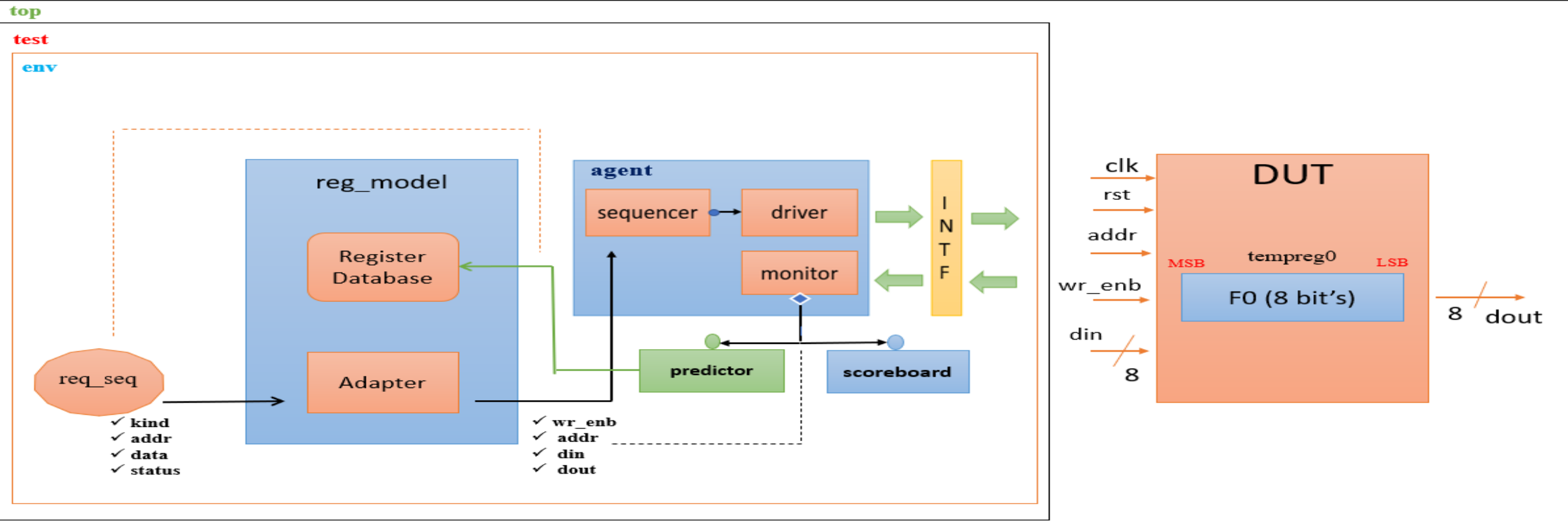
Structure of predictor



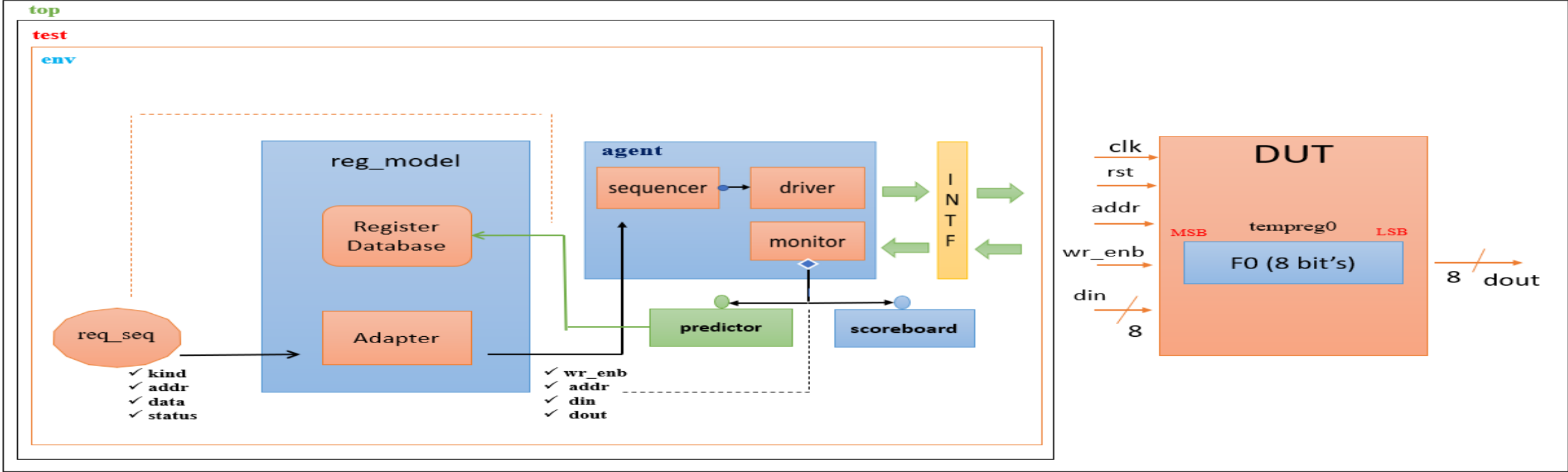
Concept of explicit prediction



Concept of explicit prediction with example



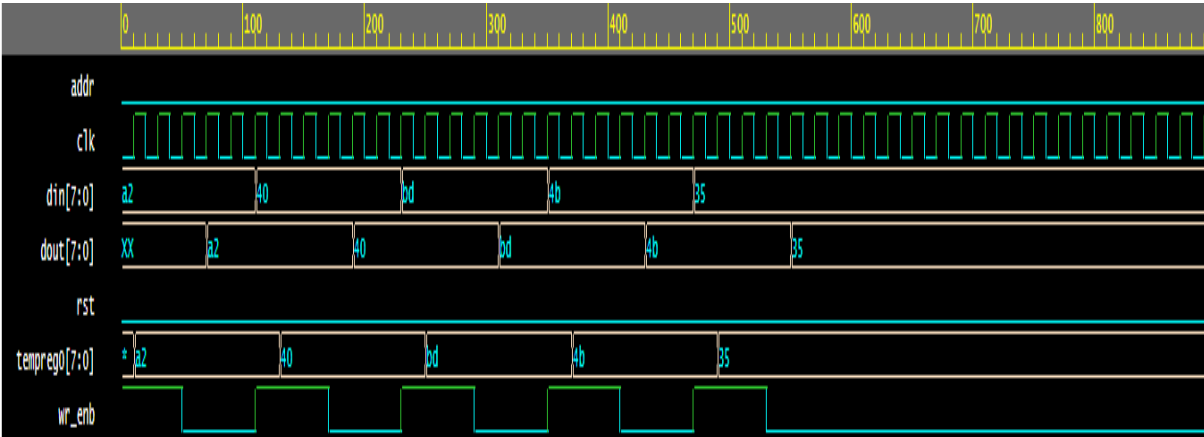
Functional Coverage for Register in SV- UVM RAL



```
# COVERGROUP COVERAGE
#
# =====
# | Covergroup | Hits | Goal / | Status |
# | | | At Least | |
# |=====|
# | TYPE /reg0/f0_cg | 100.000% | 100.000% | Covered |
# |=====|
# | INSTANCE <UNNAMED1> | 100.000% | 100.000% | Covered |
# |-----|
# | COVERPOINT <UNNAMED1>::\\1 \\ | 100.000% | 100.000% | Covered |
# |-----|
# | bin range0 | 7 | 1 | Covered |
# | bin range1 | 4 | 1 | Covered |
# | bin range2 | 4 | 1 | Covered |
# |=====
```

Part 16

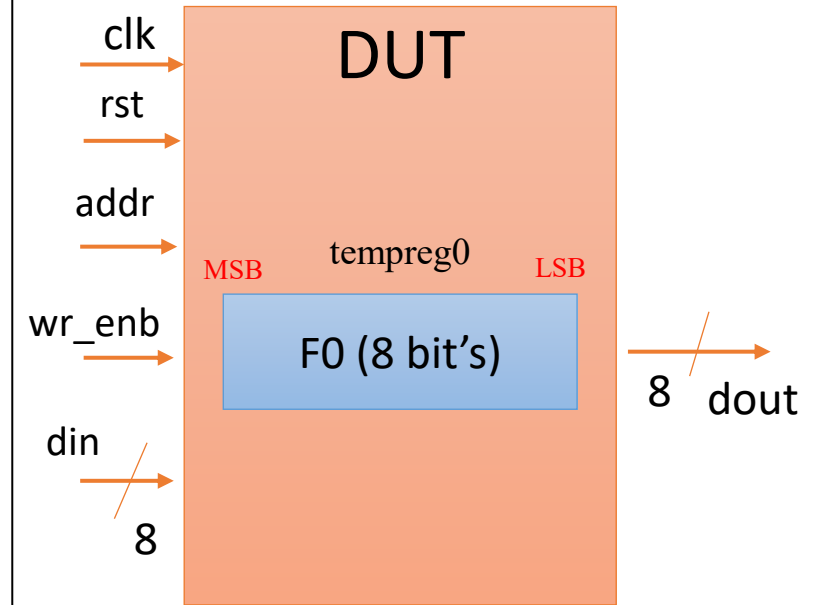
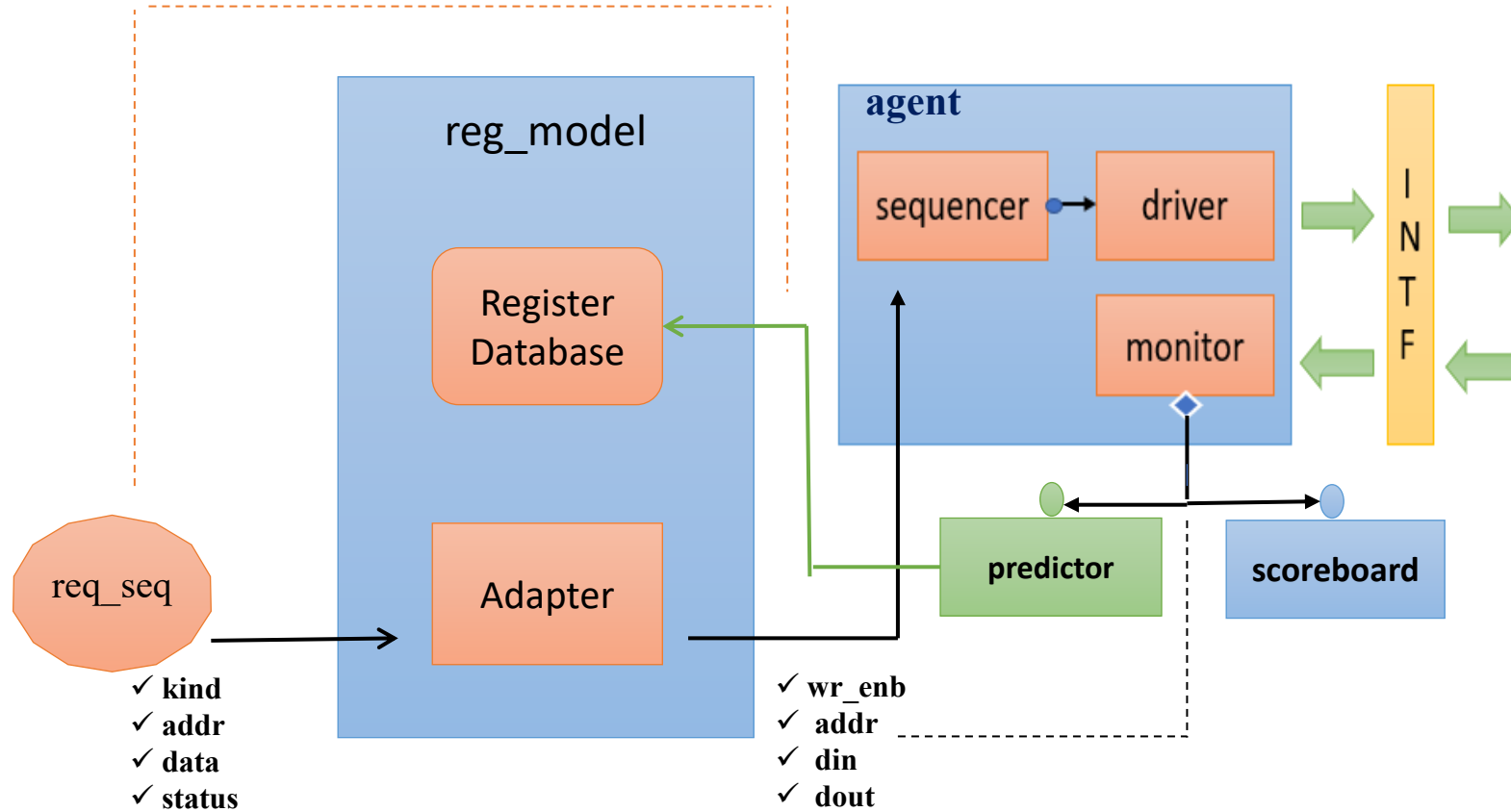
Munsif M. Ahmad



top

test

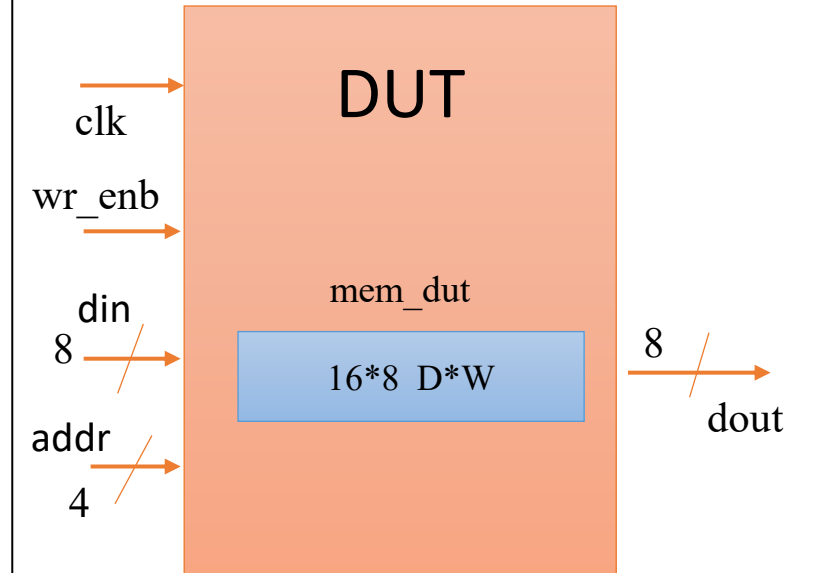
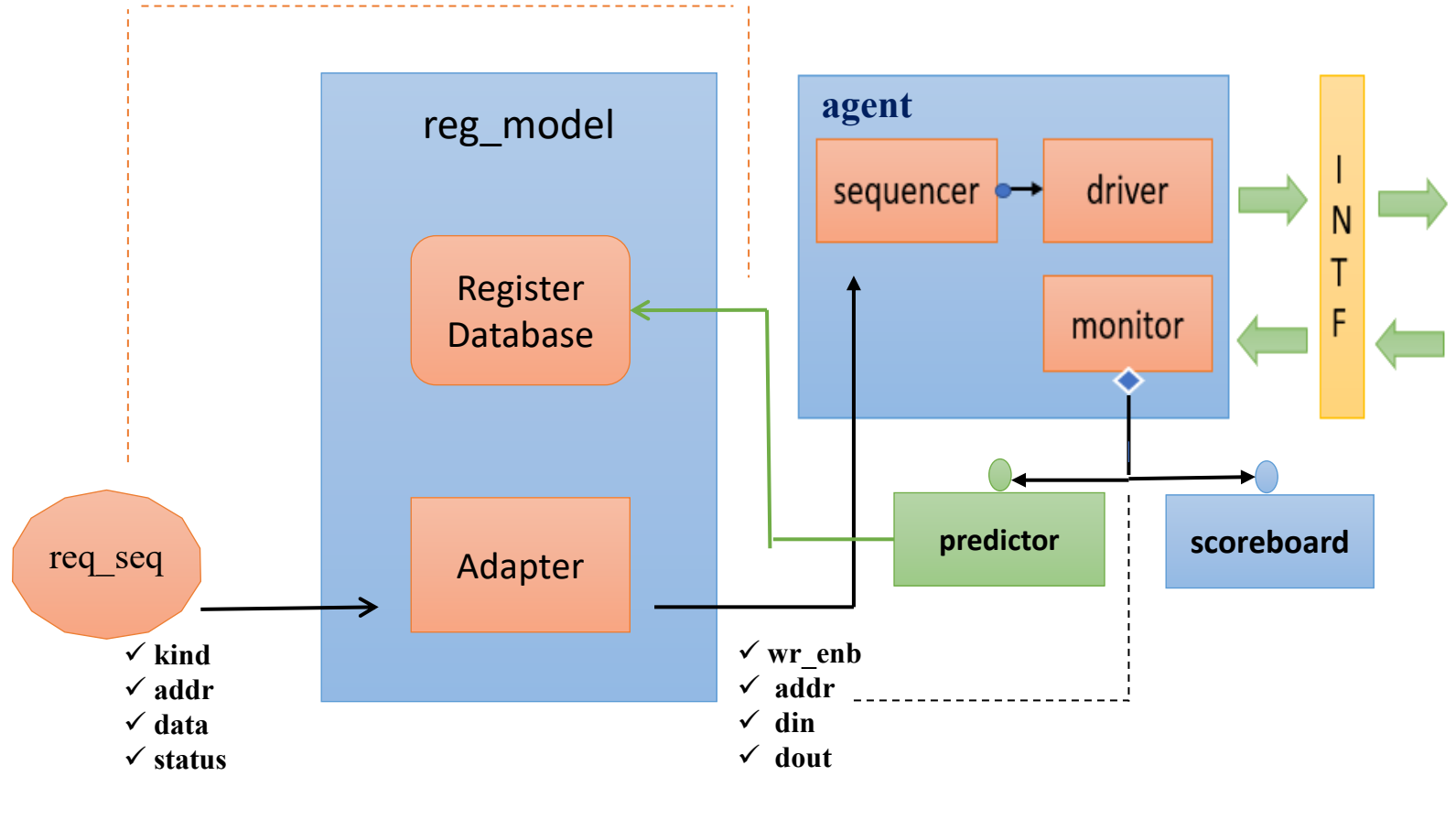
env



top

test

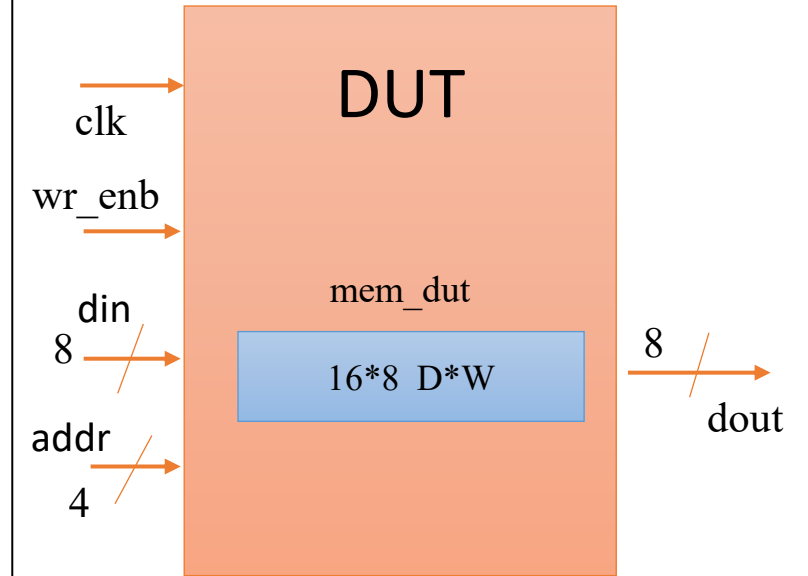
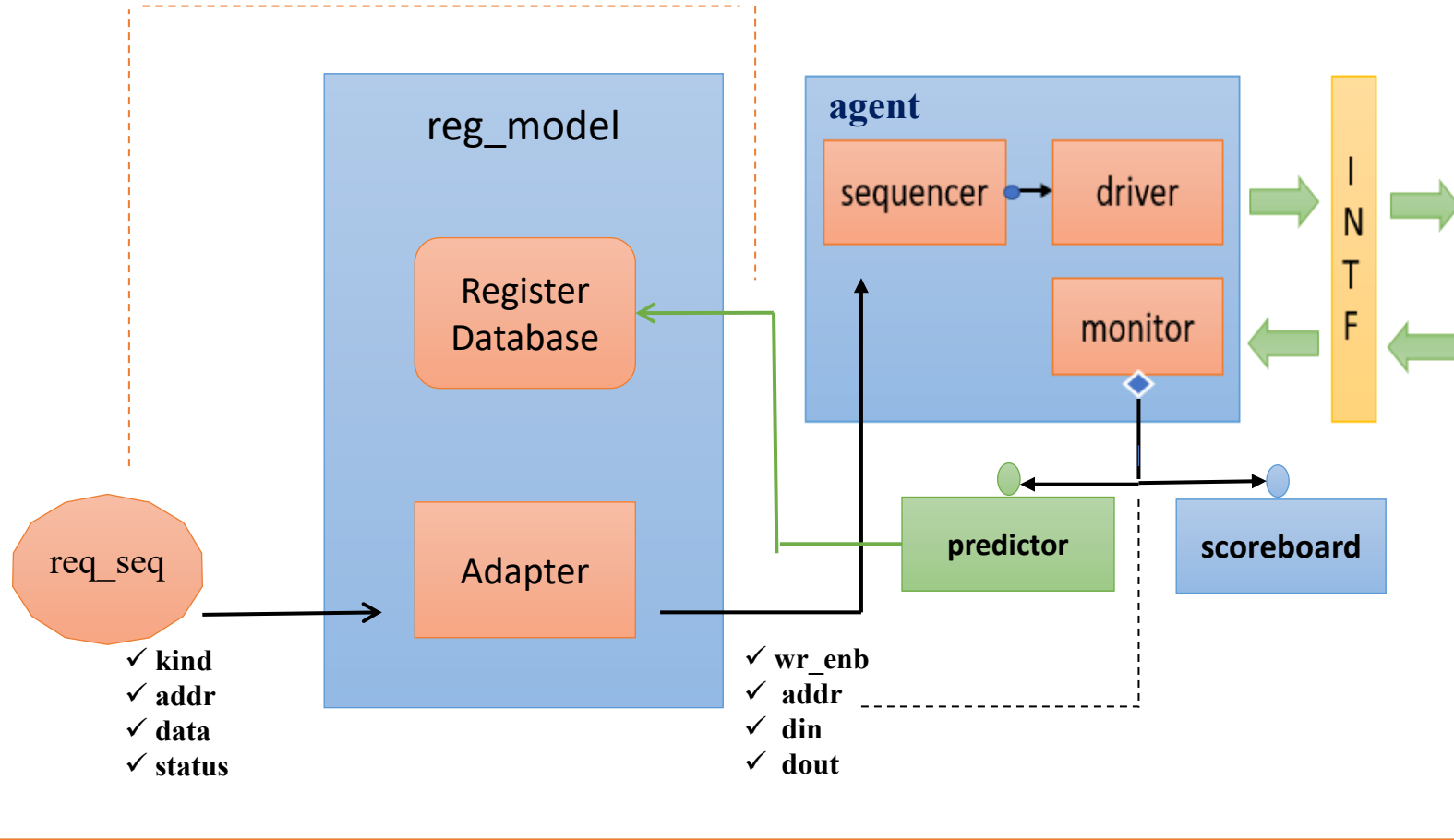
env



top

test

env



register declaration in RAL

```
// ----- register class -----

class reg0 extends uvm_reg;

    `uvm_object_utils(reg0)

    rand uvm_reg_field f0;                                // all the fields of register, here only one filed i.e f0

    function new(input string name = "reg0");
        super.new(name, 8, UVM_NO_COVERAGE);              // 8 -> width(size) of reg, & no seperate coverage for it
    endfunction: new

    // build method for fields creation and configuration method for configuration information for register

    virtual function void build();

        f0 = uvm_reg_field::type_id::create("f0");          // create a field

        f0.configure(                                       // by name
            .parent(this),                                  // current class
            .size(8),                                       // width
            .lsb_pos(0),                                    // lsb postion
            .access("RW"),                                  // type of an access
            .volatile(0),                                    // volatile -> 1 means field can't change between consecutive access
            .reset(0),                                       // reset value(power on reset value for field)
            .has_reset(1),                                   // field support reset
            .is_rand(1),                                    // field can be randomize
            .individually_accessible(1)                     // field is individually accessible
        );                                                  // 9 arguments // psl avr hii

    endfunction: build

endclass: reg0
```

mem declaration in RAL

```
// ----- memory class -----  
  
class mem extends uvm_mem;  
  
    `uvm_object_utils(mem)  
  
    function new(input string name = "mem");  
        super.new(name, 16, 8, "RW", UVM_NO_COVERAGE);  
    endfunction: new  
  
endclass: mem
```

// depth(location) -> 16, 8 -> width(size),
// "access", & UVM_NO_COVERAGE

Use of the RAL model & how it is connected to DUT?

**Adapter and predictor w.r.p.t
SV – UVM RAL**

What is RAL (Register Abstraction Layer)

- UVM RAL as the name suggests, It's a high-level object-oriented abstraction layer to access DUT Register.
- If DUT has some number of Registers and Memories, so in order to verify such design/DUT effectively some innovative approach is required, Hence UVM provides a base class library for register management and their access, this approach is referred to as UVM RAL.

Adapter w.r.p.t

SV – UVM RAL

Adapter

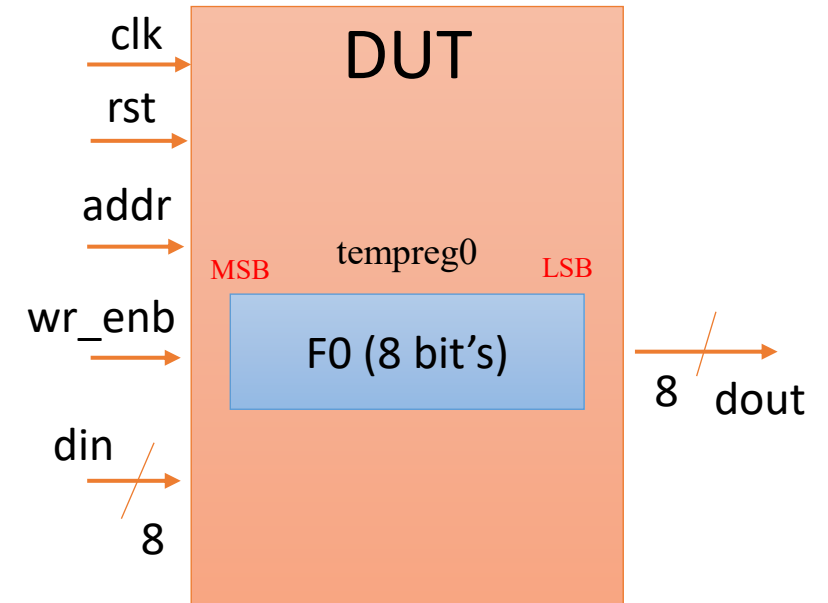
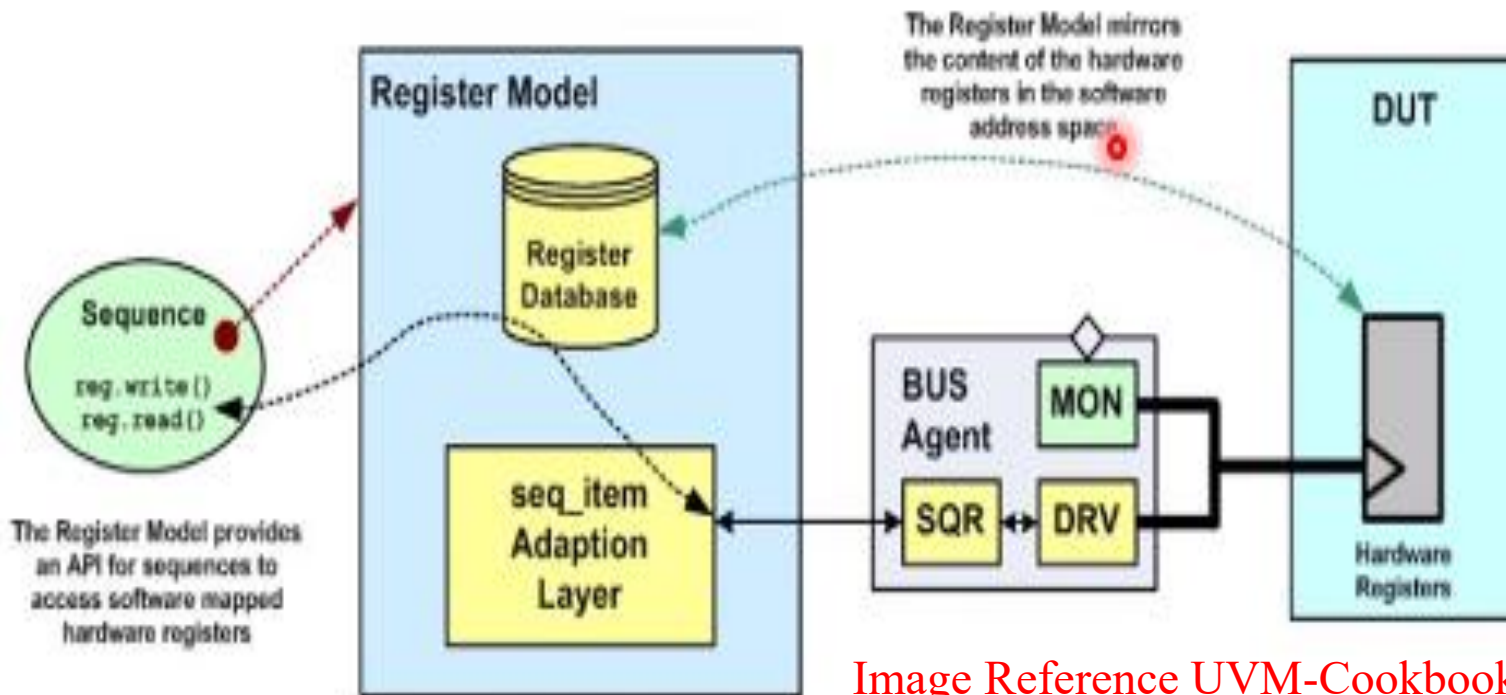
- ✓ kind
- ✓ addr
- ✓ data
- ✓ status

Reg txn

Adapter

Bus txn

- ✓ wr_enb
- ✓ addr
- ✓ din
- ✓ dout



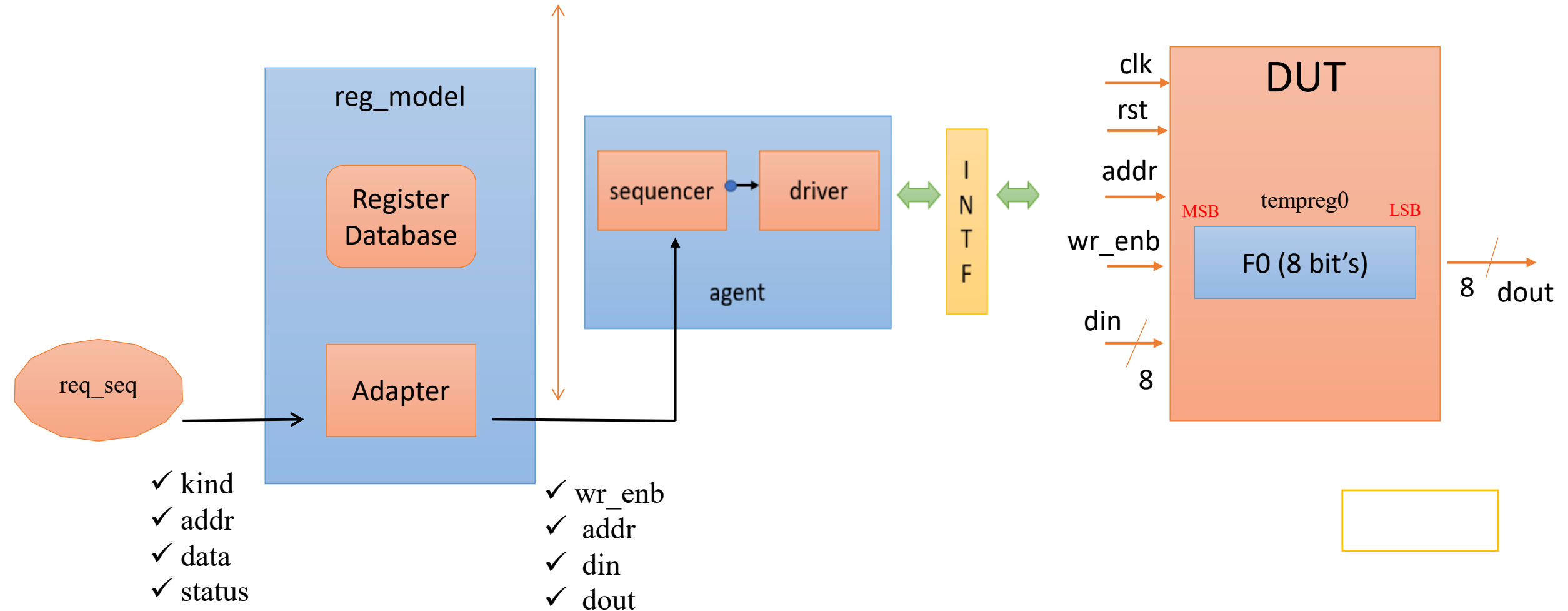
The standard structure of generic register txn type

uvm_reg_bus_op

Sr. No	Type	Property/Field	Description
1	uvm_reg_addr_t	addr	Address field, default to 64 bit's
2	uvm_reg_data_t	data	Write or Read data, default to 64 bit's
3	uvm_status_e	status	UVM_IS_OK, UVM_NO_OK, UVM_IS_X
4	uvm_access_e	kind	UVM_READ, UVM_WRITE, UVM_BRUST_WRITE, UVM_BRUST_READ
5	uvm_reg_byte_en_t	byte_en	Byte enable
6	unsigned_int	n_bits	The number of bit's being transfer

Sample env for understanding an adapter

```
reg_blk_h.default_map.set_sequencer(.sequencer(agt_h.seqr), .adapter(adapter_h));
```



```
// ----- adapter class -----
```

```
class adapter extends uvm_reg_adapter;
```

```
    `uvm_object_utils(adapter)
```

```
function new(input string name = "adapter");  
    super.new(name);  
endfunction: new
```

```
// reg2bus method:- convert reg txn to bus txn  
// return uvm_sequence_item
```

```
virtual function uvm_sequence_item reg2bus(const ref uvm_reg_bus_op rw); // 1 argument
```

```
    transaction itm = transaction::type_id::create("itm"); // instance of txn class and object creation
```

```
    itm.wr_enb = (rw.kind == UVM_WRITE) ? '1 : '0; // updating the bus txn with reg txn  
    itm.addr = rw.addr;
```

```
    if(itm.wr_enb) itm.din = rw.data;
```

```
    return itm;
```

```
endfunction: reg2bus
```

```
// bus2reg:- convert bus txn to reg txn
```

```
virtual function void bus2reg(uvm_sequence_item bus_item, ref uvm_reg_bus_op rw); // 2 arguments
```

```
    transaction itm; // instance of xtn class
```

```
    assert($cast(itm, bus_item)); // to get correct child class
```

```
    rw.kind = (itm.wr_enb) ? UVM_WRITE : UVM_READ; // updating the reg txn with bus txn  
    rw.data = itm.dout;  
    rw.addr = itm.addr;  
    rw.status = UVM_IS_OK;
```

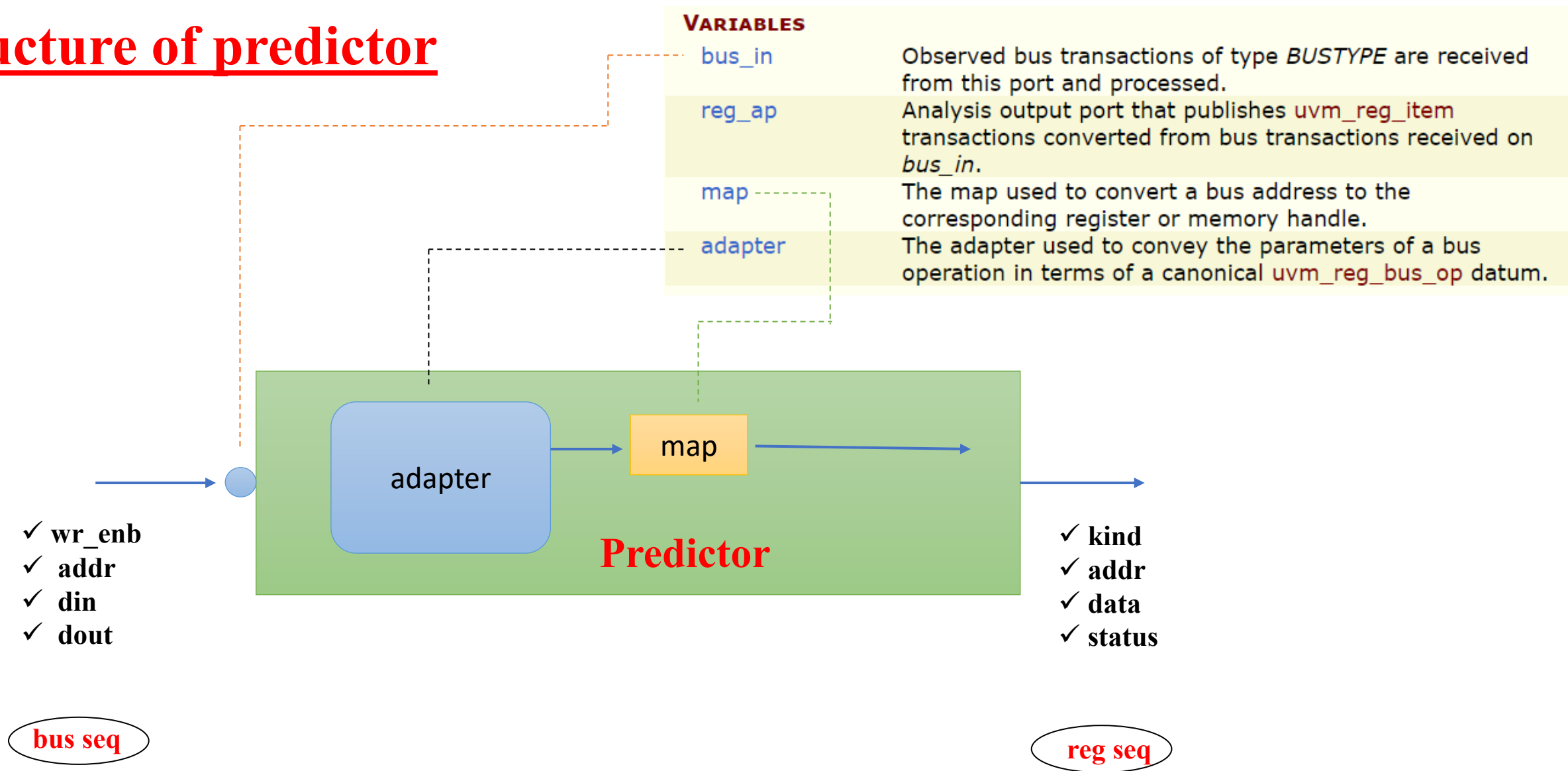
```
endfunction: bus2reg
```

```
endclass: adapter
```

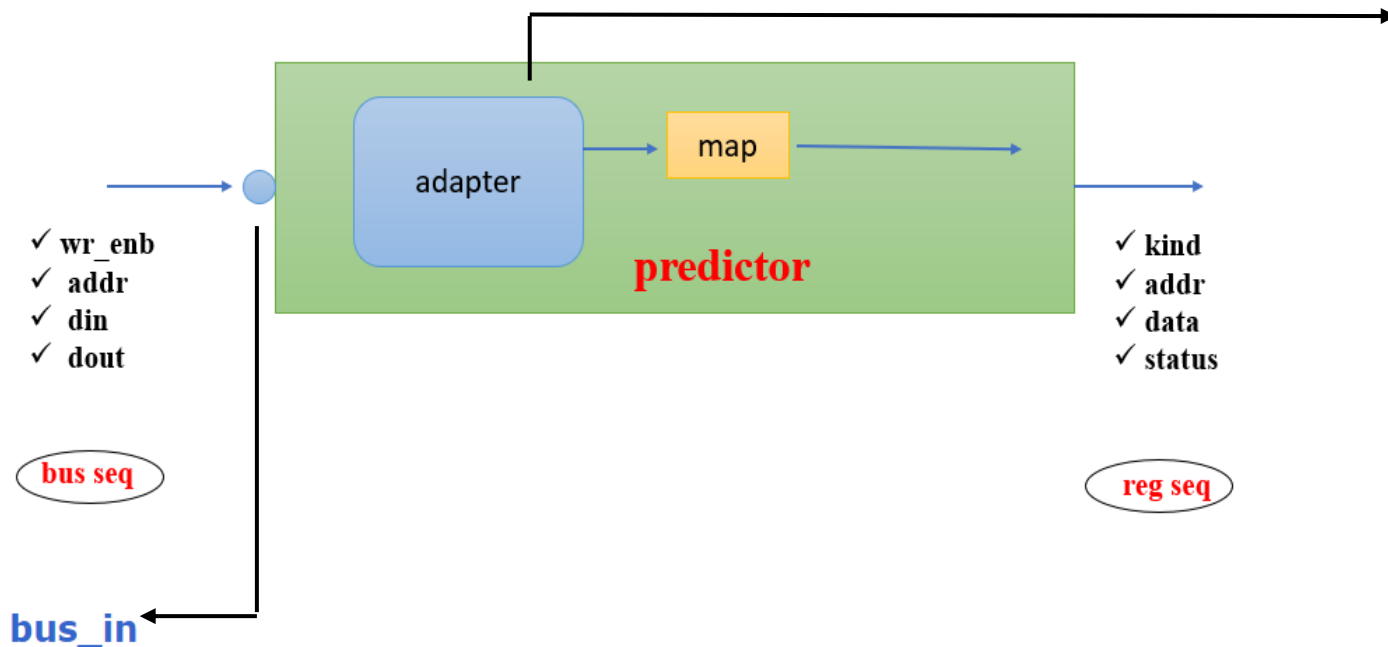
Predictor w.r.p.t SV-UVM RAL

- ☐ Auto prediction
- ☐ Explicit prediction
- ☐ Passive prediction

Structure of predictor



Structure of predictor



adapter

uvm_reg_adapter adapter

The adapter used to convey the parameters of a bus operation in terms of a canonical `uvm_reg_bus_op` datum. The `uvm_reg_adapter` must be configured before the run phase.

Observed bus transactions of type *BUSTYPE* are received from this port and processed.

For each incoming transaction, the predictor will attempt to get the register or memory handle corresponding to the observed bus address.

If there is a match, the predictor calls the register or memory's predict method, passing in the observed bus data. The register or memory mirror will be updated with this data, subject to its configured access behavior--RW, RO, WO, etc. The predictor will also convert the bus transaction to a generic `uvm_reg_item` and send it out the *reg_ap* analysis port.

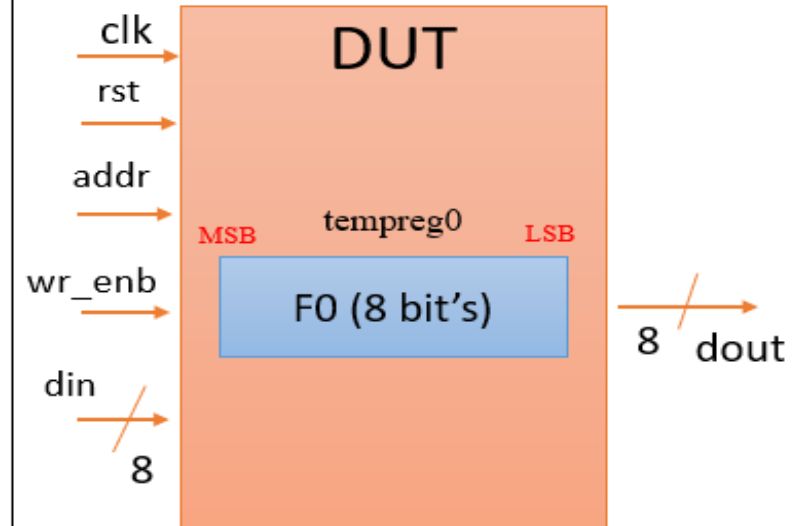
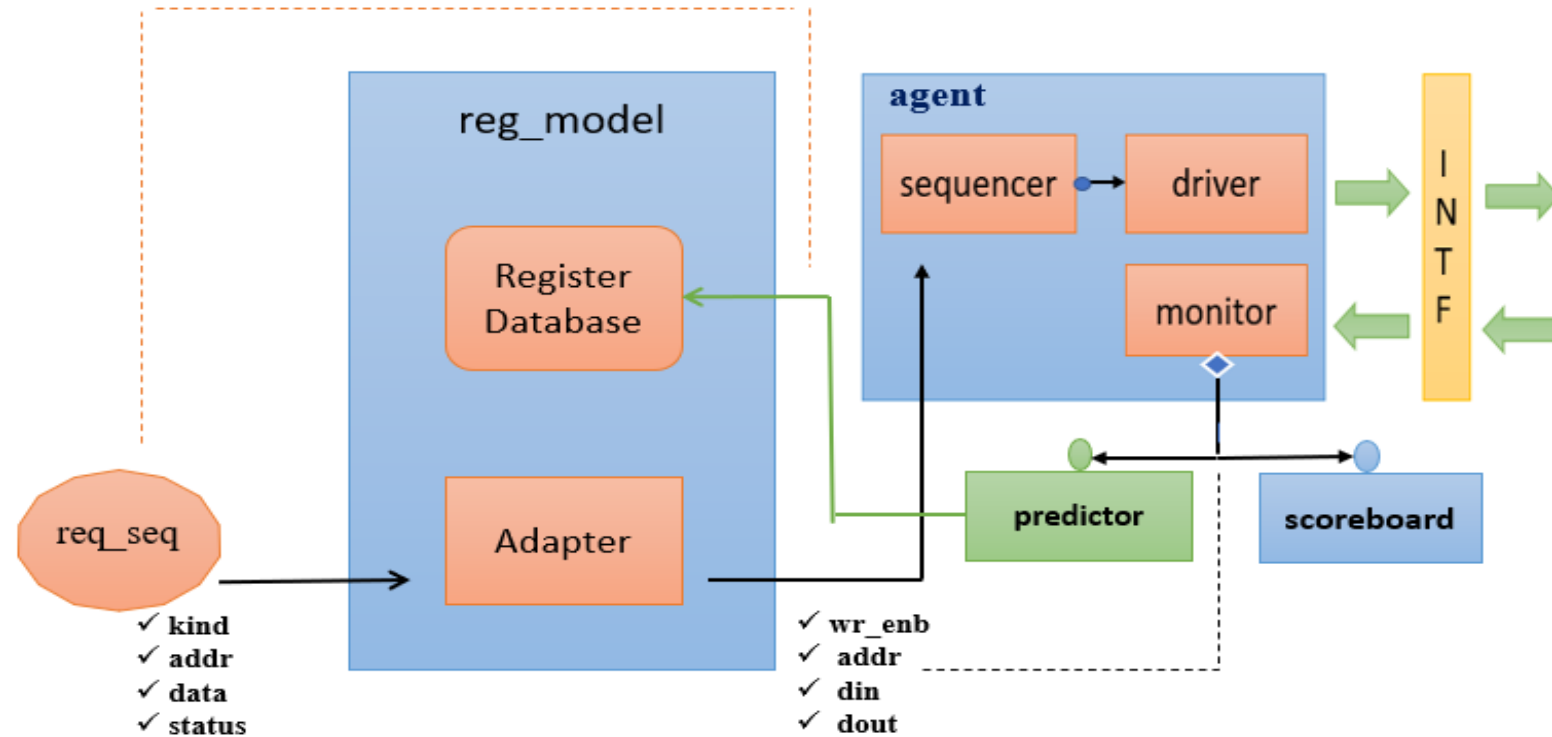
If the register is wider than the bus, the predictor will collect the multiple bus transactions needed to determine the value being read or written.

Concept of explicit prediction with example

top

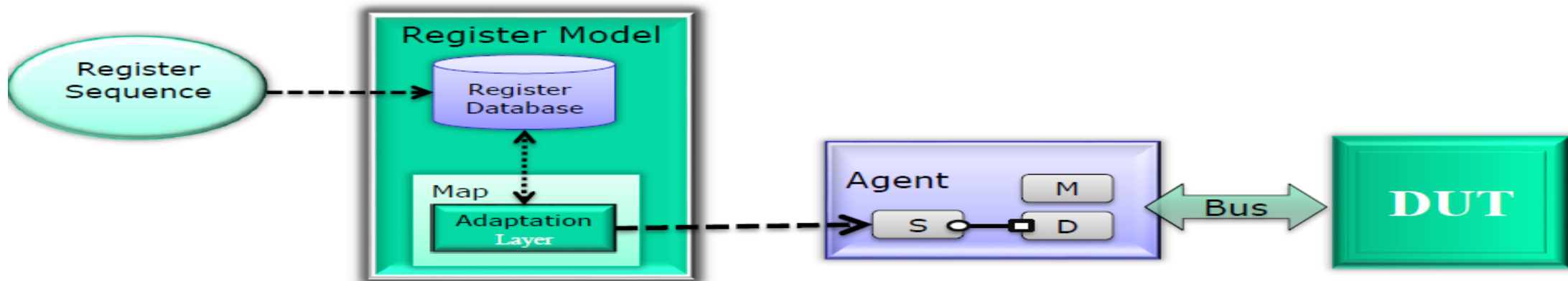
test

env



Auto prediction

- Updates Mirror Register
 - At the end of front door read and write operation
- Limitations
 - Fails to observe bus operations that does not originate from the register



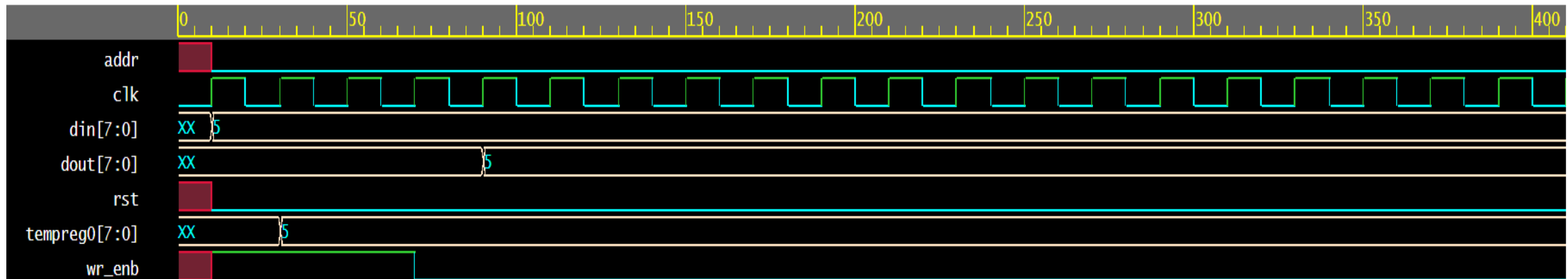
It's important to note that auto-prediction is a feature provided by the UVM RAL framework, and its availability and implementation details may vary depending on the specific RAL implementation or extensions used in your project.

Auto prediction example with simple write and read methods

<https://edaplayground.com/x/gRZH>

<https://www.edaplayground.com/x/LFks> (With separate files for all classes)

```
# KERNEL: UVM_INFO /home/runner/testbench.sv(127) @ 50: uvm_test_top.env.agt_h.drv [driver] DATA WRITE wr_data 5
# KERNEL: UVM_INFO /home/runner/testbench.sv(343) @ 50: uvm_test_top.env.agt_h.seqr@reg_seq [register_sequence] After write method Desired value is 5, & Mirrored value is 5
# KERNEL: UVM_INFO /home/runner/testbench.sv(135) @ 110: uvm_test_top.env.agt_h.drv [driver] DATA READ rd_data 5
# KERNEL: UVM_INFO /home/runner/testbench.sv(356) @ 110: uvm_test_top.env.agt_h.seqr@reg_seq [register_sequence] After read method Desired value is 5, & Mirrored value is 5, read_out is 5
# KERNEL: UVM_INFO /home/build/vlib1/vlib/uvm-1.2/src/base/uvm_objection.svh(1271) @ 410: reporter [TEST_DONE] 'run' phase is ready to proceed to the 'extract' phase
# KERNEL: UVM_INFO /home/runner/testbench.sv(549) @ 410: uvm_test_top [test] -----
# KERNEL: UVM_INFO /home/runner/testbench.sv(550) @ 410: uvm_test_top [test] ----- TEST PASS -----
# KERNEL: UVM_INFO /home/runner/testbench.sv(551) @ 410: uvm_test_top [test] -----
```



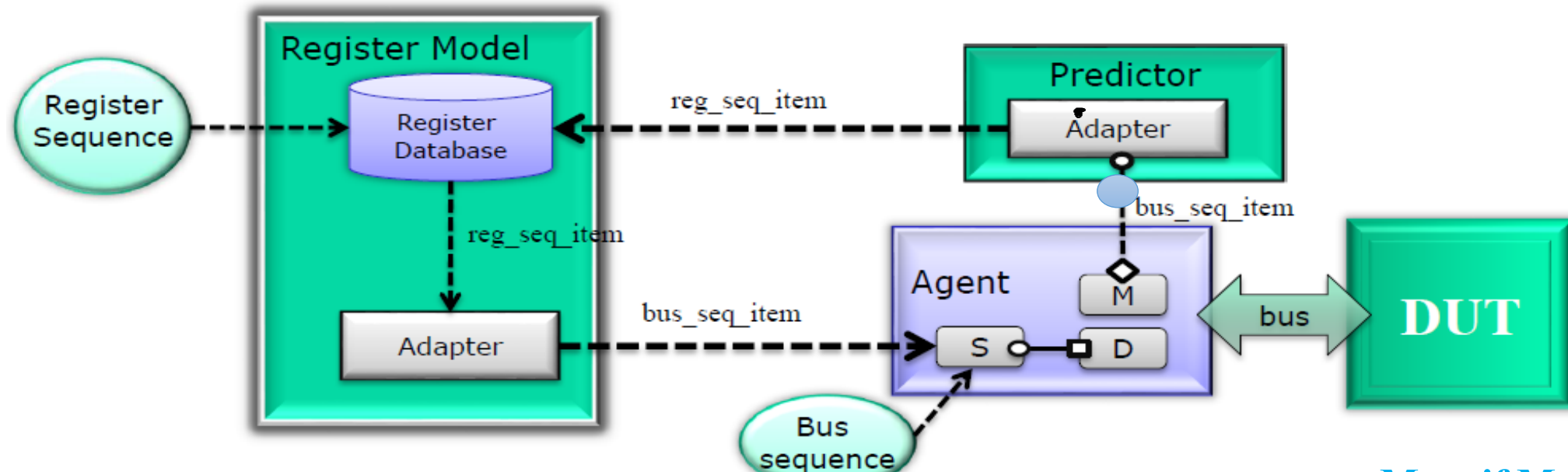
Explicit prediction (Recommended Approach)

Implicit prediction is turned off

The predictor receives the bus operations observed by a connected bus monitor

Update the mirror and desired registers at the end of the front door read and write cycles

Has the ability to observe bus operations that does not originate from the register sequence



Explicit prediction example with simple write and read methods

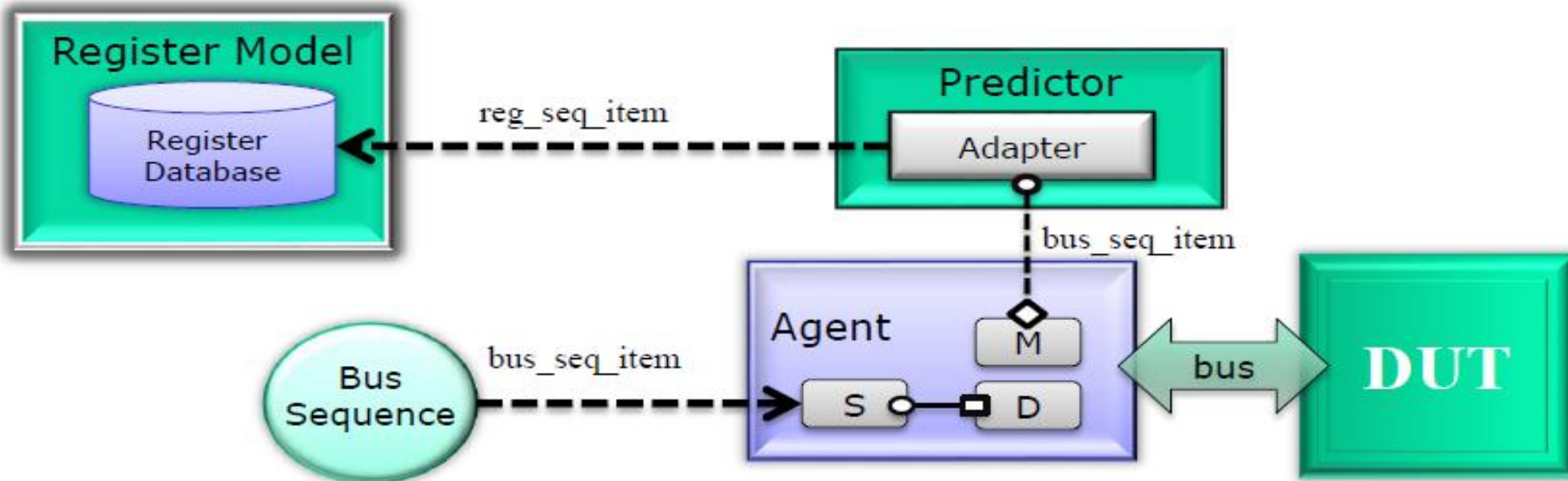
- <https://edaplayground.com/x/WTn8>
- <https://www.edaplayground.com/x/Pbb4> (Explicit prediction with separate files)
- Link for explicit register predictor (Verification Academy)
- https://verificationacademy.com/verification-methodology-reference/uvm/docs_1.2/html/files/reg/uvm_reg_predictor-svh.html
- https://verificationacademy.com/verification-methodology-reference/uvm/docs_1.2/html/files/reg/uvm_reg_predictor-svh.html#uvm_reg_predictor
- **Assignment 1:- Try with bus sequence as well as reg sequence.**
- **Assignment 2:- Why there is no dv and mv pair for mem?**

Explicit prediction example with simple write and read methods

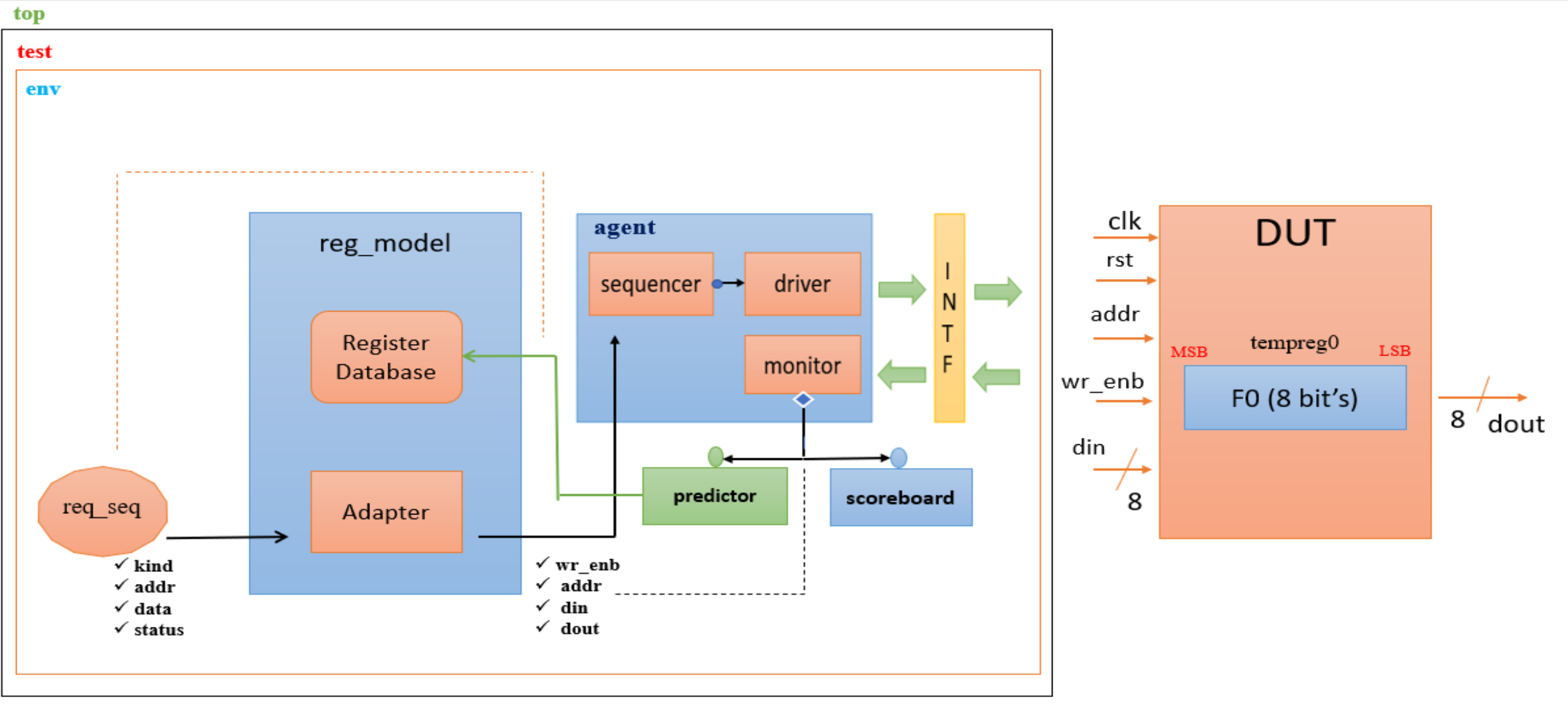
- <https://edaplayground.com/x/WTn8>
- <https://www.edaplayground.com/x/Pbb4> (Explicit prediction with separate files)
- Link for explicit register predictor (Verification Academy)
- https://verificationacademy.com/verification-methodology-reference/uvm/docs_1.2/html/files/reg/uvm_reg_predictor-svh.html
- https://verificationacademy.com/verification-methodology-reference/uvm/docs_1.2/html/files/reg/uvm_reg_predictor-svh.html#uvm_reg_predictor

Passive prediction

Passive integration only requires the integration of the register model with the bus monitor. Because the register model is not integrated with a bus sequencer, it cannot be used to read and write register and memories in the DUT, only to track and verify their current value.



Concept of explicit prediction with example



Assignments

- Assignment 1 :- Adapter class for APB and AHB protocols ?
- Assignment 2 :- What will happen if `rw.status = UVM_NOT_OK;` in `bus2reg` of an adapter class for `set_auto_predict(1)` & for explicit prediction when `set_auto_predict(0)` ?
- Assignment 3 :- Try with bus sequence as well as reg sequence (For understanding explicit prediction)?
- Assignment 4 :- Why there is no any desired and mirrored values for mem class ?
- Assignment 5 :- Hand shaking mechanism between `req_sequence` and driver ?